Gaussian Filtering and Edges

Pablo Mesejo
pmesejo@go.ugr.es
Universidad de Granada
Departamento de Ciencias de la Computación e Inteligencia Artificial



En esta presentación vamos a profundizar un poco en el filtrado gaussiano y en la detección de bordes.

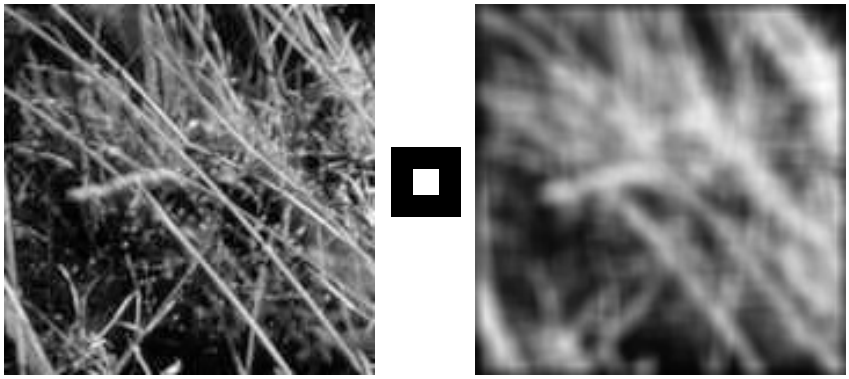
Index

- Smoothing with a box filter vs with a Gaussian filter
- Gaussian kernel discretization
- Gaussian properties and filter separability
- Computing derivatives: high-frequency features (high-pass filters)
  - First-order derivatives
  - Second-order derivatives (the Laplacian of Gaussian)
- The Canny Edge Detector

Más concretamente, vamos a comenzar comparando el suavizado producido por un *box filter* con el de un filtro gaussiano. Recordemos que un *box filter* utiliza un kernel en el que todos los coeficientes tienen el mismo valor, por lo que todas las posiciones del vecindario pesan por igual. Para que actúe como un filtro de suavizado, normalizamos dividiendo por la suma de los coeficientes; por ejemplo, en un kernel de 3×3 dividiríamos entre 9.

Después profundizaremos un poco en la discretización de un kernel gaussiano, algo que ya hemos visto en la Práctica 1. Veremos también algunas propiedades del filtro gaussiano, como su separabilidad. A continuación, nos adentraremos en el cálculo de derivadas, es decir, en filtros paso alto orientados a resaltar cambios rápidos de intensidad y, por tanto, componentes de alta frecuencia. Estudiaremos derivadas de primer y segundo orden y finalizaremos con el popular detector de bordes de Canny.

Smoothing with box filter

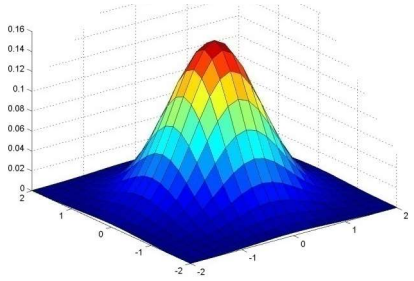


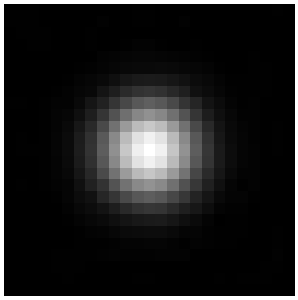
Source: D. Forsyth

En esta diapositiva vemos la imagen de entrada a la izquierda, el filtro empleado en el centro y la imagen suavizada a la derecha. El filtro se representa como un cuadrado blanco sobre fondo negro: dentro del soporte del kernel todos los coeficientes tienen el mismo valor, de ahí que aparezca uniforme, mientras que fuera de ese soporte los valores son cero. Por tanto, se trata de un *box filter*.

A la derecha observamos claramente el efecto de suavizado: la imagen aparece emborronada, pero además pueden apreciarse ciertos artefactos o estructuras espurias, como una especie de malla sutil.

Gaussian kernel





An isotropic (circularly symmetric) filter

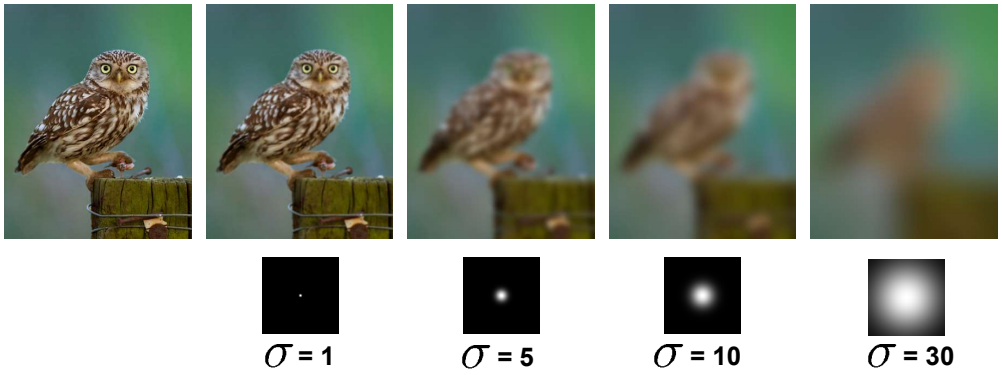
$$G_\sigma = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

- The coefficients are a 2D Gaussian.
- Gives more weight to the central pixels and less weight to the neighbors.
- The farther away the neighbors, the smaller the weight.

En esta diapositiva vemos ahora un kernel gaussiano. Arriba a la izquierda aparece una visualización tridimensional del kernel, mientras que abajo a la izquierda se muestra la expresión matemática de la función gaussiana bidimensional.

Arriba a la derecha vemos ese mismo kernel desde una vista cenital, de forma similar a como representábamos el *box filter* en la diapositiva anterior. A partir de esta representación podemos destacar varias propiedades. Por un lado, se trata de un filtro isotrópico, es decir, circularmente simétrico. Sus coeficientes vienen determinados por la gaussiana 2D y asignan mayor peso al píxel central y pesos progresivamente menores a los vecinos conforme aumenta la distancia al centro.

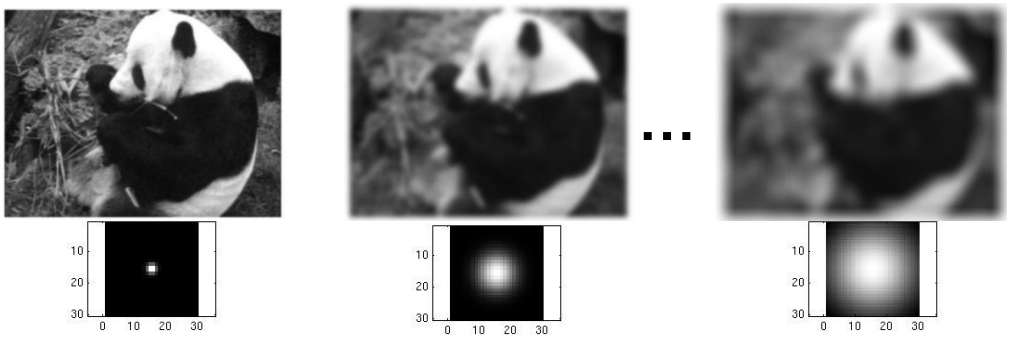
Gaussian filters



En esta diapositiva vemos el efecto del suavizado a medida que aumentamos sigma. La desviación típica sigma controla la extensión espacial de la gaussiana: cuanto mayor es sigma, más lentamente decrecen sus coeficientes al alejarnos del centro y, por tanto, mayor es la región de la imagen que contribuye de manera significativa al filtrado. Como consecuencia, aumenta el grado de suavizado o emborronamiento. Esto es precisamente lo que observamos: con sigma=1 se perciben ligeras diferencias, mientras que con sigma=30 la imagen aparece fuertemente emborronada.

Smoothing with a Gaussian

Parameter σ is the “scale” / “width” / “spread” of the Gaussian kernel and controls the amount of smoothing.




```
sigma=0
for ksize in np.arange(3, 9, 2):
        kernel=cv2.getGaussianKernel(ksize, sigma)
        outim=cv2.filter2D(im,-1,kernel@kernel.T)
        cv2.imshow(outim)
```

If it’s non-positive, it’s computed from ksize

If you use Google Colab, `from google.colab.patches import cv2_imshow`

When ddepth=-1, the output image will have the same depth/precision as the source

We create the 2D kernel from the 1D kernel

Depth combinations

Input depth ( )	Output depth (ddepth)
CV_8U	-1/CV_16S/CV_32F/CV_64F
CV_16U/CV_16S	-1/CV_32F/CV_64F
CV_32F	-1/CV_32F
CV_64F	-1/CV_64F

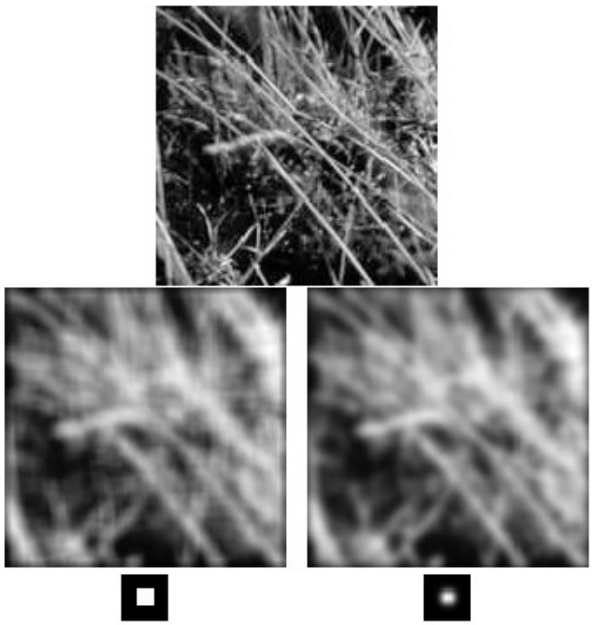
En esta diapositiva redundamos un poco en lo comentado en la anterior e introducimos también una nota técnica. La idea principal es que el parámetro sigma controla la escala, anchura o dispersión del kernel gaussiano y, por tanto, la cantidad de suavizado.

En la parte inferior de la diapositiva podéis ver un fragmento de código en el que se fija sigma = 0. Esto no significa que vayamos a construir una gaussiana con desviación típica cero. En cv2.getGaussianKernel, si se proporciona un valor no positivo para sigma, OpenCV calcula automáticamente un valor de sigma a partir de ksize.

En este ejemplo recorremos distintos tamaños de kernel comenzando en 3 y avanzando de dos en dos, por lo que probamos ksize = 3, 5 y 7.

Finalmente, llamamos a cv2.filter2D. El valor -1 en el segundo argumento indica que la imagen de salida tendrá la misma profundidad o tipo numérico que la imagen de entrada. Como cv2.getGaussianKernel devuelve un kernel columna, construimos el kernel gaussiano 2D mediante kernel @ kernel.T. El operador @ realiza multiplicación matricial y, dadas las dimensiones en este caso, equivale a realizar el producto exterior entre el vector columna y su transpuesta. Después aplicamos ese kernel 2D a la imagen y visualizamos el resultado del suavizado.

Mean vs. Gaussian filtering



The **box filter** provides **edgy artifacts!**

Gaussian is a better low-pass filter → it does not create high-frequency artifacts.

7 of 89

En esta diapositiva comparamos el resultado del filtrado de medias con el filtrado gaussiano. En la parte superior vemos la imagen original de entrada. En la parte inferior izquierda aparece el resultado de aplicar un *box filter*, mientras que en la parte inferior derecha vemos el resultado de aplicar un kernel gaussiano.

El filtro de caja puede producir ciertos artefactos alrededor de los bordes, mientras que el filtro gaussiano presenta un comportamiento más suave. En este sentido, podemos considerar el filtro gaussiano un mejor filtro de paso bajo, ya que su respuesta en frecuencia es también gaussiana y no presenta este tipo de artefactos.

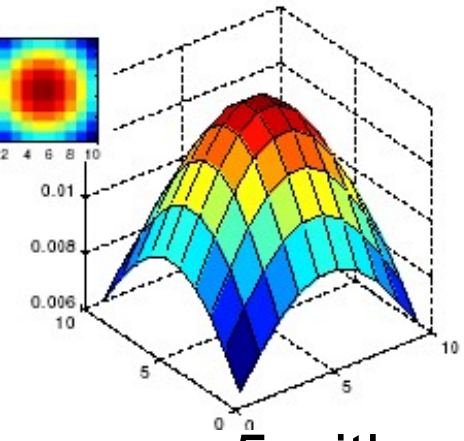
Gaussian filter

Which of th
discretizations/ap
below do you th

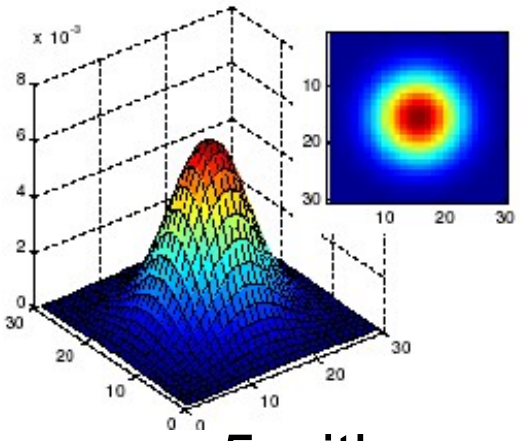
hat parameters matter here?

Size of kernel or mask

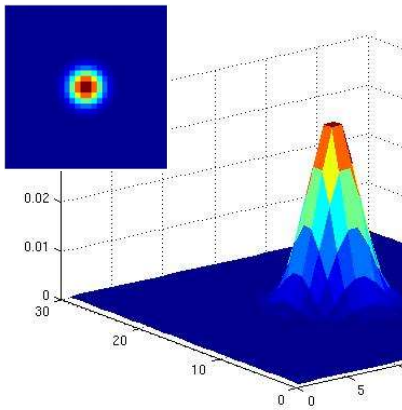
Standard deviation (sigma) of Gaussian determines extent of smoothin



σ = 5 with 11 x 11 kernel



σ = 5 with 31 x 31 kernel



σ = 2 w
31 x 3
kerne

Cuando pensamos en un filtro gaussiano, hay dos parámetros principales que nos interesan. El primero, común a cualquier kernel, es el tamaño de la máscara. El segundo, específico del filtro gaussiano, es sigma, la desviación típica, que controla la dispersión de la gaussiana y, por tanto, la extensión espacial del suavizado.

Ambos parámetros están relacionados. A la izquierda vemos, por ejemplo, sigma=5 con un kernel de 11×11. En este caso, el soporte del kernel es demasiado pequeño para una gaussiana tan ancha, por lo que estamos truncando de forma importante sus colas y obtenemos una discretización incompleta.

En el extremo contrario, a la derecha tenemos sigma=2 con un kernel de 31×31. Aquí estamos incluyendo muchos coeficientes extremadamente próximos a

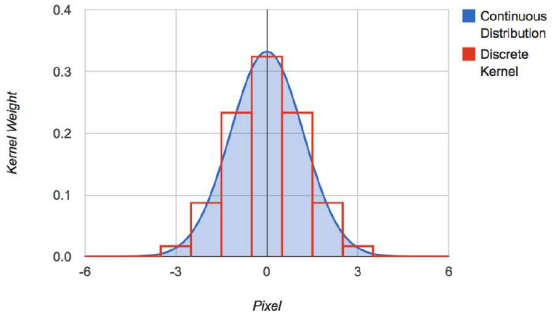
cero. Eso implica realizar multiplicaciones adicionales durante el filtrado que apenas contribuyen al resultado, por lo que el kernel es innecesariamente grande.

En el centro tenemos sigma=5 con un kernel de 31×31, una combinación mucho más razonable: el soporte es suficientemente amplio para representar bien la gaussiana sin introducir una cantidad excesiva de coeficientes prácticamente nulos.

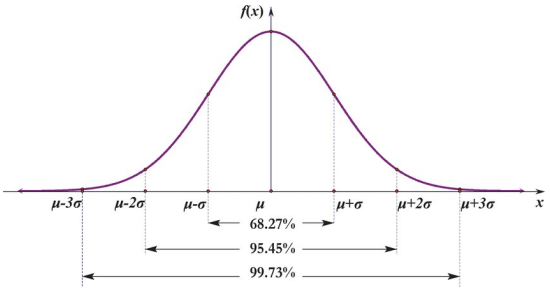
Gaussian discretization

We want to discretize a Gaussian.

There are different approaches to this discretization. Here, we will consider one of them.



We know that **almost all values are within 3 standard deviations of the mean** → It makes sense to use, **by default, 3σ.**



A la hora de discretizar una gaussiana existen distintas aproximaciones y estrategias. En las siguientes diapositivas vamos a ver una de ellas, pero conviene tener presente que existen otras alternativas, cada una con sus propias ventajas e inconvenientes.

Partimos de una función gaussiana continua, pero para construir un kernel discreto necesitamos decidir cómo muestrearla y, sobre todo, qué extensión espacial vamos a conservar. Una estrategia habitual consiste en truncarla a una cierta distancia de la media.

Como en una distribución gaussiana aproximadamente el 99,73% de la masa se encuentra dentro del intervalo mu +/- 3sigma, una elección razonable es utilizar por defecto un soporte que cubra hasta tres desviaciones típicas a cada lado de la media. De esta forma conservamos prácticamente toda la gaussiana y descartamos únicamente unas colas con peso muy pequeño.

Gaussian discretization

$$f(x,\sigma) = c \cdot e^{-\frac{x^2}{2\sigma^2}}$$

We ignore this normalizing constant ($^1/_{\sigma\sqrt{2\pi}}$)**!** The shape of the filter is the same! We later normalize the kernel, making the sum of the coefficients equal to 1

Gaussian discrete mask 1D:
$[f(-k), \dots, f(-1), f(0), f(1), \dots, f(k)], \quad k \text{ an integer}$

If σ= 1 → kernel size is 7
(ie, *k* = 3)

According to the Gaussian properties, $\min(k) \geq 3\sigma$ provides samples covering more than 99% of the area under the curve

We enforce $\sum_{i=-k}^{k} f(i) = 1$

The mask must add up to 1 → you must divide your mask by the sum of its coefficients

Why?? 🤔

10 of 89

Aquí en esta diapositiva vemos una posible forma de llevar a cabo la discretización de una gaussiana. En primer lugar, partimos de la función gaussiana e ignoramos la constante de normalización. Esa constante sirve para que, al integrar la función gaussiana en todo su dominio, el resultado sea 1 y pueda interpretarse como una función de densidad de probabilidad. En nuestro caso, esto no es estrictamente necesario, porque eliminar esa constante no modifica la forma relativa del filtro. Además, posteriormente vamos a normalizar el kernel discreto dividiendo todos sus coeficientes por la suma total, de manera que la suma final sea 1. Por tanto, podemos ignorar inicialmente esa constante de normalización.

Para construir la máscara gaussiana discreta 1D necesitamos determinar un entero k, de manera que el tamaño total del kernel sea T = 2k + 1. Como trabajamos con posiciones discretas, evaluamos la función gaussiana en valores enteros: f(0) para la posición central, f(1), f(2), ..., f(k) hacia la derecha y, simétricamente, f(-1), f(-2), ..., f(-k) hacia la izquierda.

Por ejemplo, si sigma = 1 y queremos cubrir aproximadamente tres desviaciones típicas a cada lado de la media, elegimos el menor entero k que sea mayor o igual que 3 sigma. En este caso, 3 sigma = 3, por lo que k = 3 y el tamaño total del kernel será T = 2·3 + 1 = 7.

Finalmente, debemos normalizar el kernel para que la suma de todos sus coeficientes sea 1. Para ello dividimos cada coeficiente por la suma total de los coeficientes. Esto es importante en un filtro de suavizado para preservar aproximadamente el nivel medio de intensidad de la imagen. Si la suma de los coeficientes fuese mayor que 1, tenderíamos a aumentar el brillo de la imagen y podríamos llegar a saturar valores; si fuese menor que 1, tenderíamos a oscurecerla.

Gaussian discretization

We can go from σ to mask size (T), or viceversa, in order to get the Gaussian mask

If T is the mask size, T = 2·k+1 → k = (T-1)/2 → (T-1)/2 = 3·σ → 2·[3·σ] + 1 = T

Let's see an example. We have T=5:

- We'd have 5 positions: [f(-2), f(-1), f(0), f(1), f(2)]
- We compute the suitable sigma: (T-1)/2 = 3·σ → (5-1)/2 = 3·σ → 2 = 3·σ → σ = 2/3 = 0.67
- We compute the values of the mask substituting $x = \{-2, -1, 0, 1, 2\}$ on $f(x) = e^{-\frac{x^2}{2 \cdot 0.67^2}}$

Once we have σ and *k* we can discretize the mask by applying the Gaussian function (or its derivatives)

Note: the discretization of a continuous function is just an approximation. There is no single way of doing it! See, for instance, Stan Birchfield's chapter 5.2.4 on constructing Gaussian kernels, or the way in which OpenCV operates when using `getGaussianKernel()`.

Y ahora vemos la operación inversa. En la diapositiva anterior, dado un valor de sigma, calculábamos un tamaño de kernel adecuado. Aquí partimos del tamaño del kernel y estimamos el sigma correspondiente siguiendo el mismo criterio de truncamiento en ±3 sigma.

Si el tamaño de la máscara es T = 2k + 1, podemos despejar k = (T - 1) / 2. Como hemos decidido que k cubra aproximadamente 3 sigma, hacemos k = 3 sigma. Por tanto, sigma = (T - 1) / 6.

Por ejemplo, si T = 5, tenemos cinco posiciones, desde -2 hasta 2, centradas en 0. En este caso k = 2 y, aplicando el criterio anterior, sigma = 2/3, aproximadamente 0.67.

Una vez conocidos sigma y el tamaño de la máscara, simplemente evaluamos la función que queremos discretizar en las posiciones enteras correspondientes. En este ejemplo calcularíamos f(-2), f(-1), f(0), f(1) y f(2) utilizando sigma = 0.67, obteniendo así nuestro kernel gaussiano 1D discretizado.

El mismo procedimiento puede aplicarse a las derivadas de la gaussiana, aunque en ese caso hay algunos matices relativos a la normalización que veremos más adelante.

La relación sigma = (T - 1) / 6 no es una relación matemática universal entre sigma y el tamaño del kernel; resulta específicamente de nuestra decisión de truncar la gaussiana en ±3 sigma.

Gaussian discretization: finite approximation

Convolving a box filter with itself yields an approximation to a Gaussian kernel (easy way to construct a Gaussian kernel):

$$\frac{1}{2}[1 \quad 1] * \frac{1}{2}[1 \quad 1] = \frac{1}{4}[1 \quad 2 \quad 1]$$

$$\frac{1}{4}[1 \quad 2 \quad 1] * \frac{1}{4}[1 \quad 2 \quad 1] = \frac{1}{16}[1 \quad 4 \quad 6 \quad 4 \quad 1]$$

These are the odd rows of the binomial (Pascal's) triangle:

k=1, σ^2=0.5, σ=0.7

k=2, σ^2=1, σ=1

$$\begin{array}{ccccc} & & 1 & & \\ & 1 & & 1 & \\ \boxed{1} & & \boxed{2} & & \boxed{1} \\ 1 & & 3 & & 3 & & 1 \\ \boxed{1} & & \boxed{4} & & \boxed{6} & & \boxed{4} & & \boxed{1} \end{array}$$

$(2k+1)^{th}$ row approximates Gaussian with $\sigma^2=k/2$

Check Stan Birchfield's book (Chapter 5.2) for more information regarding the binomial and trinomial triangle to generate Gaussian approximations

Otra forma de construir una aproximación discreta a una gaussiana consiste en utilizar filtros binomiales. Existen distintas estrategias para discretizar funciones gaussianas, y esta es una alternativa especialmente sencilla.

La idea parte de que la convolución repetida de kernels muy simples da lugar progresivamente a una forma que aproxima una gaussiana. En particular, si partimos del kernel [1, 1] y lo convolucionamos repetidamente consigo mismo, obtenemos los coeficientes del triángulo de Pascal. Normalizando esos coeficientes para que sumen uno obtenemos filtros binomiales que aproximan kernels gaussianos.

Por ejemplo, al convolucionar [1, 1]/2 consigo mismo obtenemos [1, 2, 1]/4, un kernel de tamaño 3. Si este nuevo kernel vuelve a convolucionarse consigo mismo obtenemos [1, 4, 6, 4, 1]/16, de tamaño 5. Estos coeficientes corresponden a filas del triángulo de Pascal.

Gaussian Properties

- Remove “high-frequency” components from the image (*low-pass filter*). All values are positive and sum up to 1.
  - Images become smoother.
- Completely described by 1st (mean) and 2nd (variance) order statistics (moments).
- The Fourier Transform of a Gaussian function is also a Gaussian

$$f(x) = e^{-\frac{x^2}{2\sigma^2}} \quad \Longrightarrow \quad \mathcal{F}[f(x)] = \sqrt{2\pi}\sigma e^{-2\pi^2\sigma^2\omega^2}$$

13 of 89

La función gaussiana tiene una serie de propiedades especialmente ventajosas, y ahí reside buena parte del motivo por el que se utiliza tan ampliamente. En primer lugar, atenúa las componentes de alta frecuencia de una imagen, es decir, actúa como un filtro de paso bajo y contribuye a suavizarla.

Por otro lado, su forma queda completamente determinada por solo dos parámetros: la media y la varianza, o equivalentemente la media y la desviación típica. En ese sentido, la gaussiana queda completamente caracterizada por sus momentos de primer y segundo orden.

Finalmente, presenta otras propiedades muy convenientes. Por ejemplo, la transformada de Fourier de una función gaussiana es también una gaussiana, algo que veremos con más detalle en próximas clases y que permite una interpretación especialmente limpia de su comportamiento en el dominio de la frecuencia.

Gaussian Properties

- Closed under convolution (convolution of two Gaussians is another Gaussian)



  - When you convolve a Gaussian function (G_1) with another Gaussian function (G_2), the result is still a Gaussian function (G)
  - Convolving twice with Gaussian kernel of width σ is the same as convolving once with kernel of width $\sqrt{\sigma^2 + \sigma^2} = \sqrt{2\sigma^2} = \sigma\sqrt{2}$
  - In general, n repeated convolutions with a Gaussian of width σ are equivalent to a single convolution with a Gaussian of width $\sigma\sqrt{n}$

$$G(x, y|\sigma) = G\left(x, y \left| \sqrt{\sigma_1^2 + \sigma_2^2}\right.\right) = G_1(x, y|\sigma_1) * G_2(x, y|\sigma_2)$$

https://en.wikipedia.org/wiki/Sum_of_normally_distributed_random_variables#Proof_using_convolutions

14 of 89

Otra propiedad fundamental de la gaussiana es que es cerrada bajo convolución: la convolución de dos gaussianas vuelve a ser otra gaussiana. Si convolucionamos una gaussiana G1 de desviación típica sigma1 con otra G2 de desviación típica sigma2, obtenemos una nueva gaussiana cuya varianza es la suma de las varianzas anteriores. Es decir, sigma_resultante² = sigma1² + sigma2².

Por tanto, aplicar dos veces un filtro gaussiano con la misma desviación típica sigma es equivalente, en el caso continuo ideal, a aplicar una sola vez un filtro gaussiano con desviación típica sigma por raíz cuadrada de 2. De forma general, n convoluciones sucesivas con gaussianas de la misma sigma equivalen a una única convolución con una gaussiana de desviación típica sigma por raíz cuadrada de n.

Gaussian Properties

- Separable (efficient)
  - Factors into product of two 1D Gaussians
  - Example:

$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$

```
import scipy.signal as sp
import numpy as np

np.outer(np.array([1,2,1]), np.array([1,2,1]))

sp.convolve2d(np.array([[0,1,0],[0,2,0], [0,1,0]]),
              np.array([[0,0,0],[1,2,1], [0,0,0]]), mode='same')
```

Outer product:

$$\mathbf{u} = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_m \end{bmatrix}, \quad \mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}$$

$$\mathbf{u} \otimes \mathbf{v} = \mathbf{A} = \begin{bmatrix} u_1 v_1 & u_1 v_2 & \cdots & u_1 v_n \\ u_2 v_1 & u_2 v_2 & \cdots & u_2 v_n \\ \vdots & \vdots & \ddots & \vdots \\ u_m v_1 & u_m v_2 & \cdots & u_m v_n \end{bmatrix}$$

15 of 89

Otra propiedad de la gaussiana, posiblemente la más importante desde el punto de vista computacional, es que es separable. Esto significa que una gaussiana 2D puede expresarse como el producto de dos gaussianas 1D, una horizontal y otra vertical.

Por ejemplo, un kernel gaussiano 2D de tamaño 3×3 puede construirse a partir del vector 1D [1, 2, 1]. Si tomamos este vector como columna y lo multiplicamos por su transpuesta, obtenemos el kernel 2D correspondiente. Es decir, podemos construir el kernel mediante un producto exterior.

Esta separabilidad permite además aplicar el filtrado en dos pasos: primero convolucionar la imagen con el kernel 1D en una dirección y después con el kernel 1D en la dirección perpendicular. El resultado es equivalente al filtrado 2D completo, pero con un coste computacional mucho menor.

Separability of the Gaussian

$$G_\sigma(x,y) = \frac{1}{2\pi\sigma^2}\exp^{-\frac{x^2+y^2}{2\sigma^2}}$$

$$= \left(\frac{1}{\sqrt{2\pi}\sigma}\exp^{-\frac{x^2}{2\sigma^2}}\right)\left(\frac{1}{\sqrt{2\pi}\sigma}\exp^{-\frac{y^2}{2\sigma^2}}\right)$$

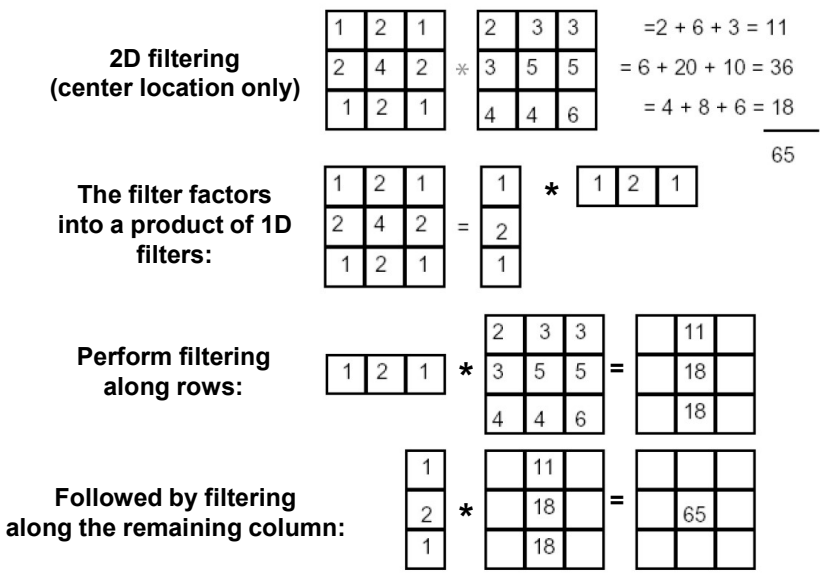
The 2D Gaussian can be expressed as the product of two functions, one a function of x and the other a function of y

In this case, the two functions are the (identical) 1D Gaussian

Source: D. Lowe

En esta diapositiva visualizamos un poco mejor, desde el punto de vista matemático, la propiedad de separabilidad de la gaussiana. Vemos que una gaussiana bidimensional puede expresarse exactamente como el producto de dos gaussianas unidimensionales: una dependiente de x y otra dependiente de y. Esta factorización es precisamente la que nos permite sustituir una convolución 2D por dos convoluciones 1D sucesivas.

Separability example



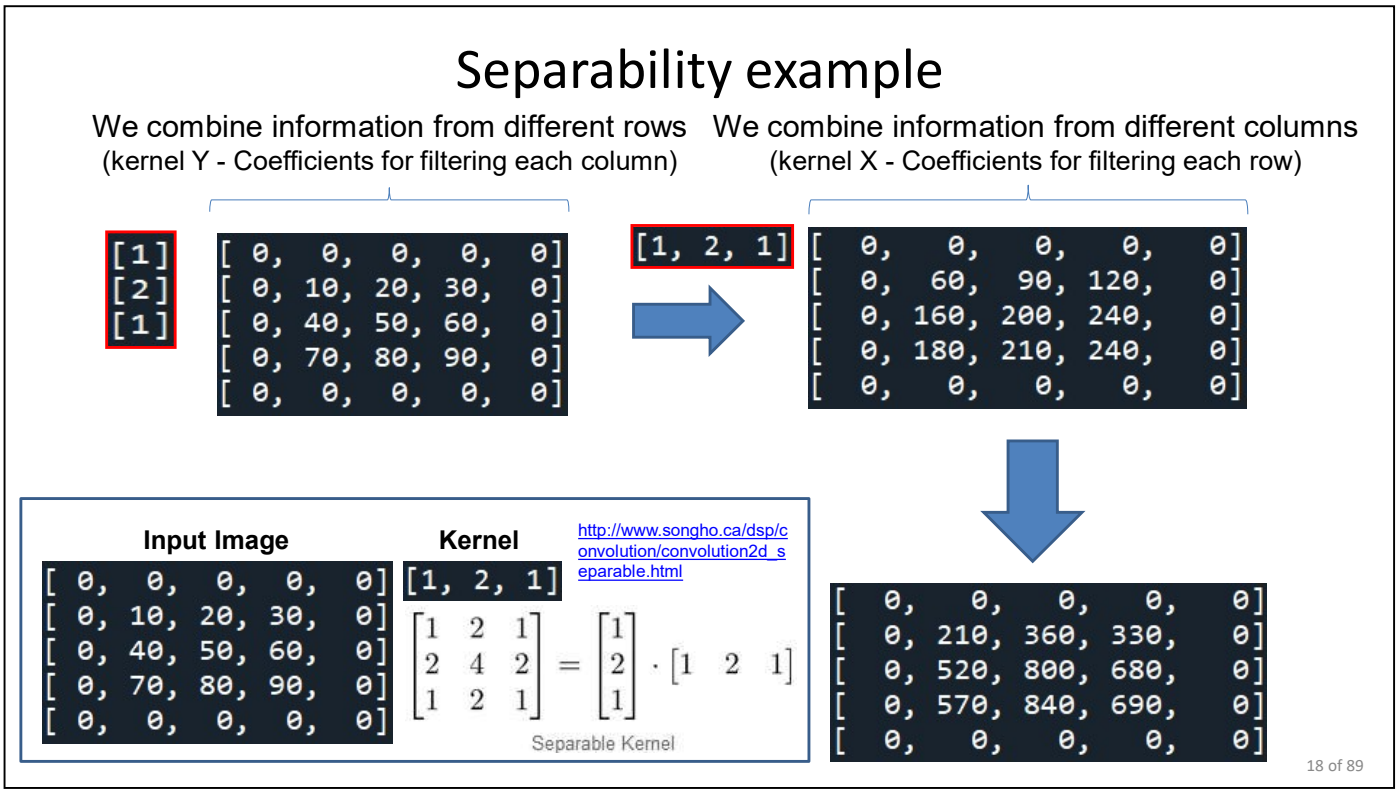
Source: K. Grauman

En esta diapositiva vemos un ejemplo concreto de separabilidad. Queremos convolucionar un kernel 2D de tamaño 3×3 con una región de la imagen también de tamaño 3×3. Como estamos calculando únicamente una posición de la imagen de salida, el resultado de esta operación es un único valor.

Si realizamos directamente la convolución 2D, multiplicamos cada valor de la imagen por el coeficiente correspondiente del kernel y sumamos todos los resultados. En este ejemplo obtenemos 11 + 36 + 18 = 65. Para ello necesitamos 9 multiplicaciones y 8 sumas.

Sin embargo, este kernel 2D es separable y puede factorizarse en dos kernels 1D. Por tanto, podemos aplicar primero una convolución 1D en una dirección y, sobre el resultado, una segunda convolución 1D en la dirección perpendicular. El resultado final es exactamente el mismo.

Este ejemplo sirve principalmente para visualizar cómo funciona la separabilidad. De hecho, si contamos únicamente las operaciones necesarias para calcular este único píxel de salida, la versión separable requiere 12 multiplicaciones y 8 sumas, por lo que aquí no observamos ninguna ventaja computacional. La ventaja aparece al filtrar una imagen completa, porque los resultados intermedios de la primera convolución 1D se reutilizan para calcular muchos píxeles de salida.



Y aquí vemos un ejemplo concreto de separabilidad. Partimos de la imagen de 3×3 que aparece arriba a la izquierda. A esta imagen se le ha añadido zero padding, de manera que pasa a tener un tamaño de 5×5.

A continuación aplicamos primero un kernel 1D vertical, es decir, un vector columna. En términos prácticos, este kernel combina información procedente de distintas filas. Sobre el resultado de esa primera convolución aplicamos después el kernel 1D horizontal, que combina información procedente de distintas columnas.

Tras estas dos convoluciones 1D obtenemos el resultado final que aparece abajo a la derecha. En este caso estamos utilizando en ambas direcciones el kernel [1, 2, 1], que es un kernel gaussiano binomial no normalizado. Su producto exterior da lugar al kernel 2D [1 2 1; 2 4 2; 1 2 1].

El resultado obtenido al aprovechar la separabilidad, es decir, aplicando sucesivamente dos convoluciones 1D, es exactamente el mismo que obtendríamos aplicando directamente una única convolución 2D con el kernel [1 2 1; 2 4 2; 1 2 1].

Why is separability useful in practice?

- Separability means that a 2D convolution can be reduced to two 1D convolutions (one along rows and one along columns)
- What is the complexity of filtering an $n \times n$ image with an $m \times m$ kernel?
  - $O(n^2 m^2)$

We move the $m \times m$ filter through the whole image. So, we do $m \cdot m$ multiplications for each of the $n \cdot n$ pixels in the image: $m^2 \cdot n^2$ multiplications

- What if the kernel is separable?
  - $O(n^2 m)$

We move the $1 \times m$ filter through the whole image. So, we do m multiplications for each of the $n \cdot n$ pixels in the image (one along rows and another along columns): $m \cdot n \cdot n + m \cdot n \cdot n = 2 \cdot (m \cdot n^2)$

Example: 512x512 image, 11x11 filter
- 1 convolution 2D: 512*512*11*11 ~32M multiplications
- 2 convolutions 1D: 512*512*11 + 512*512*11 ~6M multiplications

19 of 89

¿Por qué resulta útil la separabilidad en la práctica? Como ya hemos adelantado, un kernel separable permite sustituir una convolución 2D por dos convoluciones 1D sucesivas, una en una dirección y otra en la dirección perpendicular.

Aquí es donde podemos analizar el coste computacional. Si tenemos una imagen de tamaño n×n y un kernel de tamaño m×m, para cada uno de los aproximadamente n² píxeles de la imagen necesitamos realizar m² multiplicaciones. Por tanto, la complejidad es $O(n^2m^2)$.

Si el kernel es separable, podemos aplicar primero un filtro 1D de tamaño m en una dirección y después otro filtro 1D de tamaño m en la otra. Cada pasada requiere aproximadamente m multiplicaciones por cada uno de los n² píxeles, por lo que el coste total es aproximadamente $2mn^2$. En términos de complejidad, esto se reduce a $O(n^2m)$.

Por ejemplo, para una imagen de 512×512 y un kernel de 11×11, una convolución 2D directa requiere aproximadamente 512×512×11×11, es decir, unos 31,7 millones de multiplicaciones. Utilizando separabilidad, necesitamos aproximadamente 2×512×512×11, unos 5,8 millones. Por tanto, en este ejemplo reducimos el número de multiplicaciones en un factor de aproximadamente 5,5.

Aquí estamos contando únicamente multiplicaciones. Si incluyésemos también las sumas, cambiarían las constantes, pero no la complejidad asintótica.

More ideas about separability

- Allowed because convolution is associative ($I * (v * h) = (I * v) * h$)
- Some 2D kernels can be directly decomposed into 1D kernels:

$$\frac{1}{16}\begin{bmatrix}1 & 2 & 1\\ 2 & 4 & 2\\ 1 & 2 & 1\end{bmatrix} = \frac{1}{4}\begin{bmatrix}1\\ 2\\ 1\end{bmatrix} * \frac{1}{4}\begin{bmatrix}1 & 2 & 1\end{bmatrix} = \frac{1}{4}\begin{bmatrix}1 & 2 & 1\end{bmatrix} * \frac{1}{4}\begin{bmatrix}1\\ 2\\ 1\end{bmatrix}$$

$$\frac{1}{3}\begin{bmatrix}1\\ 1\\ 1\end{bmatrix} * \frac{1}{3}\begin{bmatrix}1 & 1 & 1\end{bmatrix} = \frac{1}{9}\begin{bmatrix}1 & 1 & 1\\ 1 & 1 & 1\\ 1 & 1 & 1\end{bmatrix}$$

- A 2D kernel is *separable* iff every pair of nonzero rows/cols is linearly dependent
  - This filter
```
0  1  0
1 -4  1
0  1  0
```
is not separable:

α·[0, 1, 0] + β·[1, -4, 1] = [0, 0, 0] ↔ α = 0 Λ β = 0 → Linearly independent vectors!!

Note: any 2D-mask admits a SVD decomposition (*low-rank approximations*): sum of several separable kernels.
https://bartwronski.com/2020/02/03/separate-your-filters-svd-and-low-rank-approximation-of-image-filters/

20 of 89

Y en esta última diapositiva desarrollamos un poco más algunas ideas relacionadas con la separabilidad. En primer lugar, la separabilidad puede aprovecharse gracias a la propiedad asociativa de la convolución. Si un kernel 2D puede escribirse como la convolución de un kernel vertical V y un kernel horizontal H, entonces convolucionar la imagen con ese kernel 2D es equivalente a convolucionar primero la imagen con V y, posteriormente, el resultado con H.

Como ya hemos visto, algunos kernels 2D pueden descomponerse directamente en dos kernels 1D. En esta misma diapositiva aparecen dos ejemplos: un kernel gaussiano y un box filter.

Una tercera idea importante es que un kernel 2D es directamente separable si y solo si tiene rango 1. El rango de una matriz es el número máximo de columnas linealmente independientes, que coincide con el número máximo de filas linealmente independientes. Tener rango 1 significa que todas las filas no nulas son múltiplos unas de otras y, equivalentemente, todas las columnas no nulas son múltiplos unas de otras. Por ejemplo, el filtro [0 1 0; 1 -4 1; 0 1 0] no es directamente separable, porque las filas [0 1 0] y [1 -4 1] son linealmente independientes y, por tanto, la matriz tiene rango mayor que 1.

Ahora bien, cualquier kernel 2D puede descomponerse mediante SVD como una suma de componentes de rango 1, y cada una de esas componentes sí es separable. Por tanto, podemos implementar la convolución con el kernel original como una suma de convoluciones separables: para cada componente realizamos una convolución 1D en una dirección y otra en la dirección perpendicular, y

finalmente sumamos los resultados.

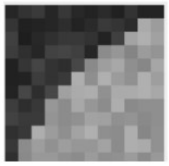
Si conservamos todas las componentes de la SVD, la representación es exacta. Si conservamos solamente las componentes asociadas a los valores singulares más importantes, obtenemos una aproximación de bajo rango que puede reducir el coste computacional.

Computing derivatives: high-frequency features (high-pass filters)

Ahora, en el siguiente bloque de diapositivas vamos a estudiar cómo podemos calcular las derivadas de una imagen y cómo estas derivadas nos permiten detectar cambios de intensidad y, en particular, bordes.

Edges

	24	25	26	27	28	29	30	31	32	33	34	35
24	15	20	23	15	22	16	14	17	13	20	14	14
25	21	13	14	15	18	14	17	23	21	17	67	61
26	25	12	16	19	22	22	20	14	25	64	69	63
27	24	19	15	26	27	25	15	55	59	52	61	58
28	13	21	27	20	18	28	54	62	66	56	53	65
29	13	14	24	20	23	68	56	69	61	57	69	62
30	14	28	18	28	63	67	68	57	67	67	69	66
31	24	18	22	59	55	66	59	55	68	54	56	56
32	26	23	23	58	55	57	64	63	57	58	54	56
33	19	22	68	61	56	64	69	56	55	68	53	57
34	22	55	62	57	62	59	59	64	55	68	67	57
35	19	54	60	60	57	55	61	56	57	61	55	60

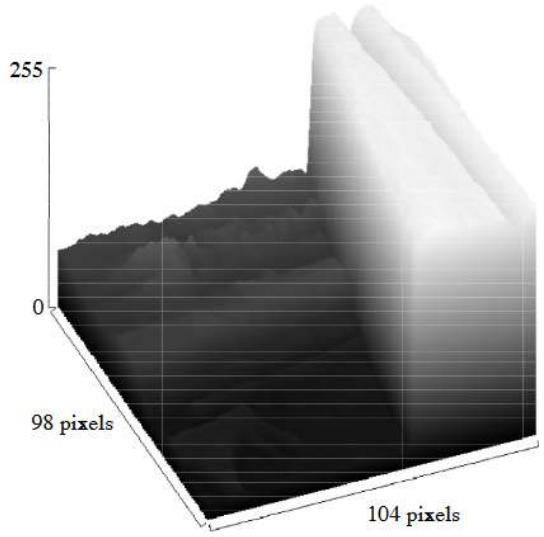


https://cs.wellesley.edu/~cs332/doc/edgeNotes.pdf

22 of 89

Lo primero que tenemos que tener presente es que un borde en una imagen corresponde, en esencia, a un cambio brusco en los niveles de intensidad. Por ejemplo, en esta diapositiva podéis ver a la derecha la imagen en escala de grises y, a la izquierda, esa misma imagen representada mediante los valores numéricos correspondientes a cada nivel de gris.

Edges



Edge: an abrupt change in pixel intensity.

Contour: a closed curve that joins edge points to represent the shape of an object.

https://www.cs.auckland.ac.nz/~rklette/TeachAuckland.html/775/pdfs/D02-Images.pdf

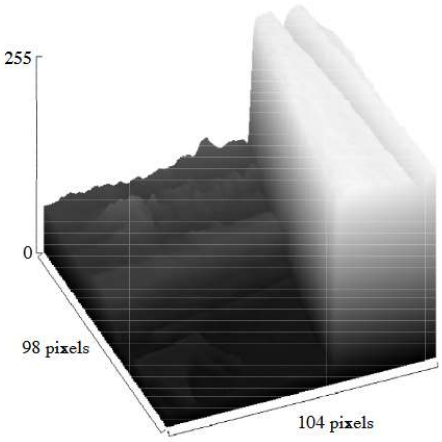
23 of 89

En esta diapositiva vemos un borde de una forma que, a mí al menos, me parece especialmente intuitiva. Como ya os comenté en una clase anterior, puede ser muy útil visualizar una imagen como si fuese un paisaje tridimensional, en el que la altura representa el nivel de intensidad. De esta forma podemos imaginar mesetas, valles o montañas dependiendo de los valores de gris de la imagen.

Desde este punto de vista, un borde en visión por computador corresponde normalmente a una transición brusca de intensidad, es decir, a una zona con una pendiente elevada en ese paisaje. Un contorno, en cambio, puede entenderse como una curva formada a partir de puntos de borde y que delimita o representa la forma de una región u objeto.

Edges and High Frequencies

- Frequency in 2D images:
  - rate of change of image intensity with respect to spatial coordinates
    - If it "involves" many pixels to make a change
      → low frequency
    - If it "involves" few pixels to make a change (i.e., the change is abrupt)
      → high frequency



24 of 89

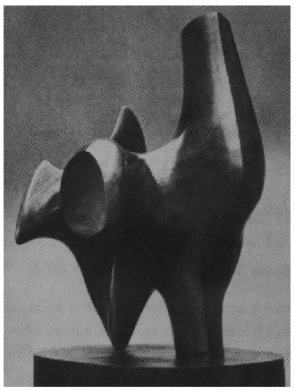
Este paisaje tridimensional también nos permite interpretar con bastante facilidad qué entendemos por frecuencia en una imagen. La frecuencia no está asociada directamente con niveles de gris altos o bajos, sino con la rapidez con la que cambian espacialmente esos niveles de intensidad.

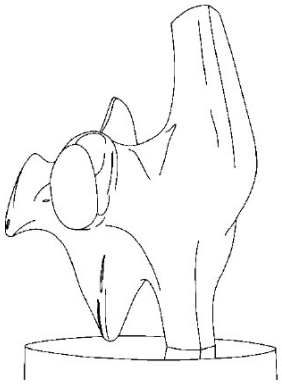
Cuando la intensidad cambia lentamente a lo largo de muchos píxeles, hablamos de bajas frecuencias espaciales. En cambio, cuando el cambio se produce de manera brusca en una distancia muy pequeña, hablamos de altas frecuencias.

Por ejemplo, en esta diapositiva vemos una región oscura en la que los niveles de gris apenas cambian; por tanto, está asociada principalmente con bajas frecuencias. Lo mismo ocurre en la meseta blanca, con valores próximos a 255: aunque la intensidad sea alta, si apenas cambia espacialmente seguimos hablando de bajas frecuencias.

En cambio, en la zona de transición entre la región oscura y la región clara aparece un cambio brusco de intensidad y, por tanto, una fuerte componente de alta frecuencia. Del mismo modo, si dentro de la propia meseta clara aparece alguna ruptura o transición abrupta, esa zona también contribuirá con componentes de alta frecuencia.

Edge detection





- **Convert a 2D image into a set of curves**
  - **Extracts salient features of the scene**
  - **More compact than pixels**

25 of 89

Una pregunta muy pertinente es por qué queremos detectar bordes y por qué, históricamente, desde los años 60 y 70, tantos investigadores e investigadoras han estado interesados en este problema.

La idea fundamental es que los bordes permiten extraer de la imagen aquellas estructuras que corresponden a cambios significativos de intensidad. En cierto modo, buscamos transformar la imagen original en una representación formada por curvas o estructuras más simples que describan aspectos relevantes de la escena.

Esta representación puede ser mucho más compacta que trabajar directamente con todos los píxeles de la imagen y, al mismo tiempo, conservar información importante sobre la forma, los límites y la organización de los objetos presentes en ella.

Importance of intensity edges



from Walther et al., Simple line drawings suffice for functional MRI decoding of natural scene categories, PNAS 2011

→ Much information is retained by the edges

S. Birchfield, Clemson Univ., ECE 847, http://www.ces.clemson.edu/~stb/ece847

Y, de hecho, si nos paramos a pensarlo, tiene mucho sentido utilizar los bordes como una representación informativa de una imagen. En esta diapositiva vemos varios sketches en los que prácticamente se ha eliminado toda la información de color y textura y se conservan únicamente trazos que representan los bordes principales de la escena.

A pesar de esa simplificación tan fuerte, seguimos siendo capaces de inferir bastante información. La primera imagen parece corresponder a un paisaje, posiblemente tropical, con una playa. La segunda parece un entorno urbano, con al menos un coche y quizá la terraza de una cafetería. La tercera recuerda a un bosque. La cuarta parece representar una carretera o autovía con vehículos. La quinta parece un paisaje montañoso y la sexta, un despacho o una oficina.

Es decir, incluso utilizando únicamente información asociada a los bordes, seguimos conservando una parte muy importante de la estructura de la escena.

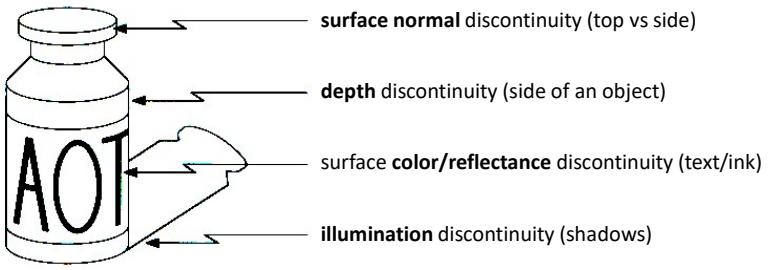
Importance of intensity edges



from Walther et al., Simple line drawings suffice for functional MRI decoding of natural scene categories, PNAS 2011

→ *Much information is retained by the edges*

Origin of edges



- Edges are caused by a variety of factors

28 of 89

Los bordes pueden originarse por causas muy diferentes. En esta diapositiva vemos algunos ejemplos. Un borde puede estar producido por una discontinuidad en la iluminación, como ocurre con una sombra. En ese caso, el cambio de intensidad no corresponde necesariamente a un cambio en el objeto, sino a cómo está iluminada la escena.

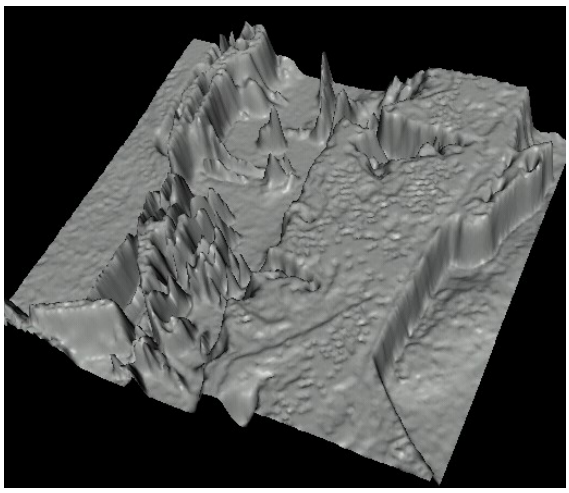
También puede estar producido por diferencias de color o reflectancia. Por ejemplo, las letras impresas sobre la superficie del frasco generan bordes porque reflejan la luz de manera distinta al fondo sobre el que están escritas.

Otra posibilidad es una discontinuidad de profundidad, por ejemplo, en el límite visible de un objeto respecto al fondo. Y también pueden aparecer bordes debido a cambios en la orientación de la superficie, es decir, en su normal. Un ejemplo sería la transición entre la parte superior del tapón y su superficie lateral.

La idea importante es que un borde en la imagen es simplemente una discontinuidad o cambio significativo de intensidad, pero ese cambio puede tener orígenes físicos muy diferentes.

Images as functions…





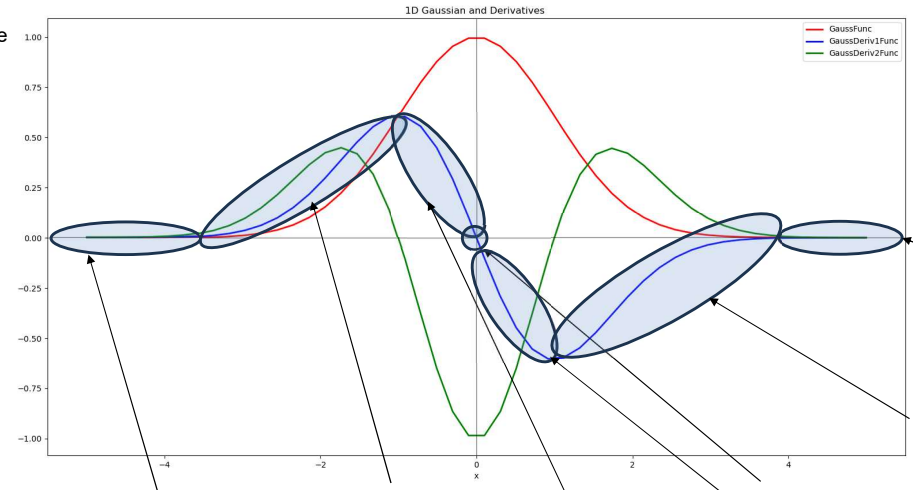
- Edges look like steep cliffs

29 of 89

Cuando visualizamos una imagen como una función o como un paisaje tridimensional, los bordes aparecen como pendientes muy pronunciadas, prácticamente como acantilados. Esto refleja precisamente que en esas zonas la intensidad cambia de manera muy rápida en una distancia espacial pequeña.

Remembering the concept of derivative

The derivative shows the **instantaneous and local sensitivity** of a **function's output with respect to small changes in the input**.



Remember that **the derivative** of a function at a chosen input value, when it exists, **is the slope of the tangent line** of the function **at that point**.

No variation → derivative=0

Decreasing negative slope

Increasing positive slope

Slope at maximum is zero (No variation) → derivative=0

No variation → derivative=0

Decreasing positive slope

Increasing negative slope

Let's pay attention to the **blue** line (first derivative of the **red** one).
This will help us to better understand the next slides.

30 of 89

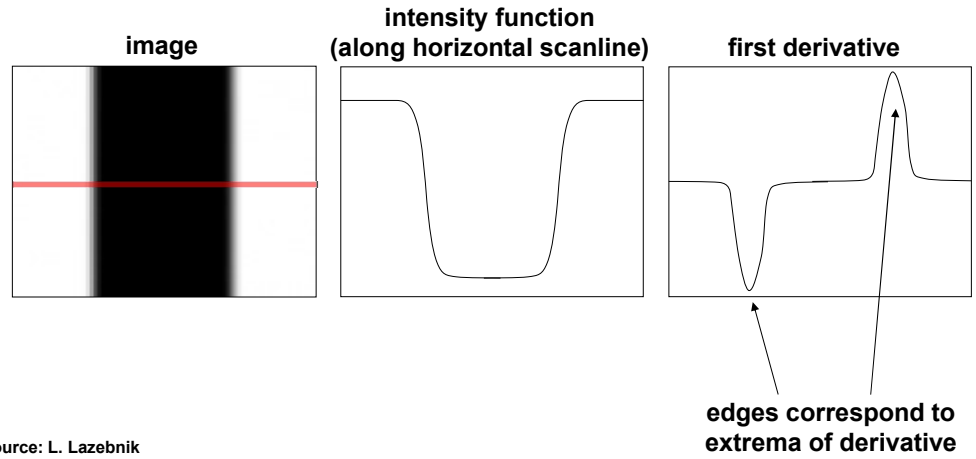
Antes de entrar en la parte más matemática, creo que es conveniente recuperar brevemente el concepto de derivada. La derivada de una función en un punto nos indica la pendiente de la recta tangente a la curva en ese punto. Por tanto, podemos interpretarla como una medida de cómo cambia la función.

En esta diapositiva, la curva roja representa una gaussiana sin la constante de normalización y la curva azul representa su primera derivada. Muy lejos del centro, la gaussiana cambia muy poco y la derivada toma valores próximos a cero. A medida que la gaussiana empieza a crecer, la pendiente es positiva y, por tanto, la derivada también es positiva. Esa pendiente alcanza un máximo y después comienza a disminuir, aunque sigue siendo positiva, hasta llegar a x = 0.

En x = 0 alcanzamos el máximo de la gaussiana. En ese punto la pendiente es cero y, por tanto, también lo es su primera derivada. A partir de ahí la gaussiana decrece, por lo que la pendiente pasa a ser negativa. Inicialmente se hace cada vez más negativa, después disminuye su magnitud y, finalmente, conforme la gaussiana vuelve a hacerse prácticamente plana, la derivada se aproxima de nuevo a cero.

Characterizing edges

- **An edge is a place of *rapid change* in the image intensity function**



Source: L. Lazebnik

31 of 89

Como decíamos, un borde es una zona en la que la intensidad de la imagen cambia rápidamente. En esta diapositiva vemos a la izquierda una barra vertical negra sobre un fondo blanco y tomamos un corte horizontal, representado por la línea roja.

Si observamos los valores de intensidad a lo largo de ese corte, obtenemos una señal como la que aparece en el centro: comenzamos con valores altos correspondientes al fondo blanco, la intensidad disminuye al entrar en la barra negra, se mantiene en valores bajos y posteriormente vuelve a aumentar al salir de la barra.

Si calculamos la primera derivada de esa señal de intensidad obtenemos algo como lo que aparece a la derecha. Los bordes se corresponden con valores extremos de la derivada, es decir, con derivadas de gran magnitud. En el borde en el que pasamos de blanco a negro aparece una derivada fuertemente negativa, mientras que en el borde en el que pasamos de negro a blanco aparece una derivada fuertemente positiva.

El signo de la derivada nos indica, por tanto, la dirección del cambio de intensidad, mientras que su magnitud nos indica lo brusco que es ese cambio.

Image derivatives

For a 2D function, $f(x, y)$, the partial derivative is

$$\frac{\partial f(x,y)}{\partial x} = \lim_{\varepsilon \to 0} \frac{f(x+\varepsilon, y) - f(x,y)}{\varepsilon}$$

For discrete data, we can approximate the derivative using **finite differences**:

$$\frac{\partial f(x,y)}{\partial x} \approx \frac{f(x+1, y) - f(x,y)}{1}$$

We compute derivatives through the **difference between intensities of adjacent pixels**.

32 of 89

Ahora que tenemos una idea más clara, a nivel conceptual, de qué es un borde, vamos a ver cómo se materializa en el procesamiento de imágenes.

Para una función continua bidimensional sabemos cómo calcular una derivada parcial. Por ejemplo, la derivada parcial de una función f con respecto a x se define como el límite, cuando epsilon tiende a cero, de la diferencia entre f evaluada en x más epsilon, y y f evaluada en x, y, dividida por epsilon.

Sin embargo, una imagen digital está formada por muestras discretas, es decir, por píxeles. Por tanto, no podemos aplicar directamente esa definición continua y lo que hacemos es aproximar la derivada mediante diferencias finitas. En términos prácticos, esto significa calcular diferencias entre intensidades de píxeles vecinos.

Image derivatives

- **How can we differentiate a digital image F[x,y]?**
  - **Take discrete derivative (finite difference)**

$$\frac{\partial f}{\partial x}[x, y] \approx F[x + 1, y] - F[x, y]$$

How would you implement this as a linear filter?

$\frac{\partial f}{\partial x}$:

1	-1	

H_x

$\frac{\partial f}{\partial y}$:

	-1	
	1	

H_y

Source: S. Seitz 33 of 89

Y ahora en esta diapositiva vemos cómo materializamos esta idea para una imagen digital. Como trabajamos con datos discretos, aproximamos la derivada mediante diferencias finitas. Por ejemplo, la derivada respecto a x de una imagen f(x,y) puede aproximarse como f(x+1,y) - f(x,y).

Esto puede implementarse directamente mediante un filtro lineal. Basta utilizar coeficientes como [1, -1] sobre píxeles consecutivos en la dirección en la que queramos calcular la derivada. De esta forma, la convolución realiza precisamente la resta entre intensidades vecinas.

Si queremos calcular la derivada en x, utilizamos el filtro en dirección horizontal; si queremos calcularla en y, utilizamos el equivalente en dirección vertical.

Image derivatives

- Odd length avoids undesirable shifting of signal

$[1 \quad \underline{-1}]$

Forward difference

$[4 \quad 7 \quad 8 \quad 1 \quad 3] * [1 \quad \underline{-1}] = [3 \quad 1 \quad -7 \quad 2 \quad 0]$

(Don't forget to flip kernel before convolving)

The position to be replaced appears underlined.

In this example, we replicate the boundary. So, the value outside the image would be 3: -1*3+1*3=0

$[\underline{1} \quad -1]$

Backward difference

$[4 \quad 7 \quad 8 \quad 1 \quad 3] * [\underline{1} \quad -1] = [0 \quad 3 \quad 1 \quad -7 \quad 2]$

- To avoid undesirable shift, use central difference kernel

$\frac{1}{2}[1 \quad \underline{0} \quad -1]$

Central difference

$h = 2$

Forward	$f'(x) \approx \frac{f(x+h) - f(x)}{h}$
Backward	$f'(x) \approx \frac{f(x) - f(x-h)}{h}$
Central	$f'(x) \approx \frac{f(x+0.5h) - f(x-0.5h)}{h}$

Example:

$[4 \quad 7 \quad 8 \quad 1 \quad 3] * \frac{1}{2}[1 \quad \underline{0} \quad -1] = \frac{1}{2}[3 \quad 4 \quad -6 \quad -5 \quad 2]$

We don't usually care about this factor.
We just want to detect differences!

Another (separable) view:

$\frac{1}{2} \cdot [1\ 1] * [1 \quad -1]$

Smoothing Differences

Generalmente utilizamos kernels de tamaño impar porque así existe una posición central bien definida y podemos hacer que los píxeles situados a ambos lados contribuyan de forma simétrica. Sin embargo, esto no significa que no podamos utilizar kernels de tamaño par. El problema es que, en ese caso, no existe un píxel central único y tenemos que decidir a cuál de las dos posiciones centrales asociamos el resultado, lo que puede introducir un desplazamiento espacial de la señal.

Por ejemplo, podemos aproximar la derivada utilizando diferencias hacia delante. Si partimos de la señal [4, 7, 8, 1, 3], la diferencia hacia delante calcula el valor siguiente menos el actual. Teniendo en cuenta que en una convolución debemos invertir el kernel, el filtro correspondiente produce las diferencias 7-4 = 3, 8-7 = 1, 1-8 = -7 y 3-1 = 2. Si utilizamos replicación en el borde derecho, obtenemos finalmente [3, 1, -7, 2, 0].

También podríamos utilizar diferencias hacia atrás. En este caso calculamos el valor actual menos el anterior. Con replicación en el borde izquierdo obtenemos [0, 3, 1, -7, 2]. Como vemos, la información sobre las diferencias es esencialmente la misma, pero aparece desplazada una posición dependiendo de si utilizamos forward difference o backward difference.

Para evitar esta asimetría podemos utilizar diferencias centrales. En lugar de comparar el píxel actual únicamente con el de un lado, comparamos los dos vecinos situados simétricamente alrededor de él. El kernel correspondiente es 1/2 por [1, 0, -1], teniendo nuevamente en cuenta la inversión propia de la convolución.

Aplicándolo a la misma señal y utilizando replicación en los bordes, antes de introducir el factor 1/2 obtenemos [3, 4, -6, -5, 2].

Este kernel también puede obtenerse convolucionando el kernel [1, 1], asociado a un pequeño suavizado, con el kernel de diferencias [1, -1], lo que produce [1, 0, -1]. El factor 1/2 proporciona la escala correcta de la aproximación de la derivada, aunque cuando nuestro objetivo es fundamentalmente localizar cambios o bordes podemos estar más interesados en las diferencias relativas que en esa escala absoluta.

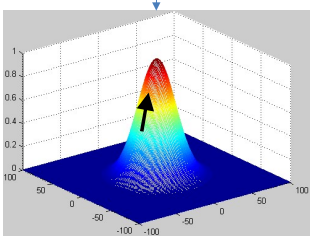
Finalmente, esta expresión puede relacionarse directamente con la fórmula de diferencias centrales. Si escribimos la derivada aproximadamente como [f(x + h/2) - f(x - h/2)] / h y tomamos h = 2, porque la distancia total entre las dos muestras comparadas es de 2 píxeles, obtenemos precisamente [f(x+1) - f(x-1)] / 2, que corresponde al kernel central 1/2 por [1, 0, -1].

Image gradient

- The *gradient* of an image: $\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right]$

The gradient points in the direction of most rapid increase in intensity





The *edge strength* is given by the **gradient magnitude**:

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

The **gradient direction** is given by:

$$\theta = \tan^{-1}\left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x}\right)$$

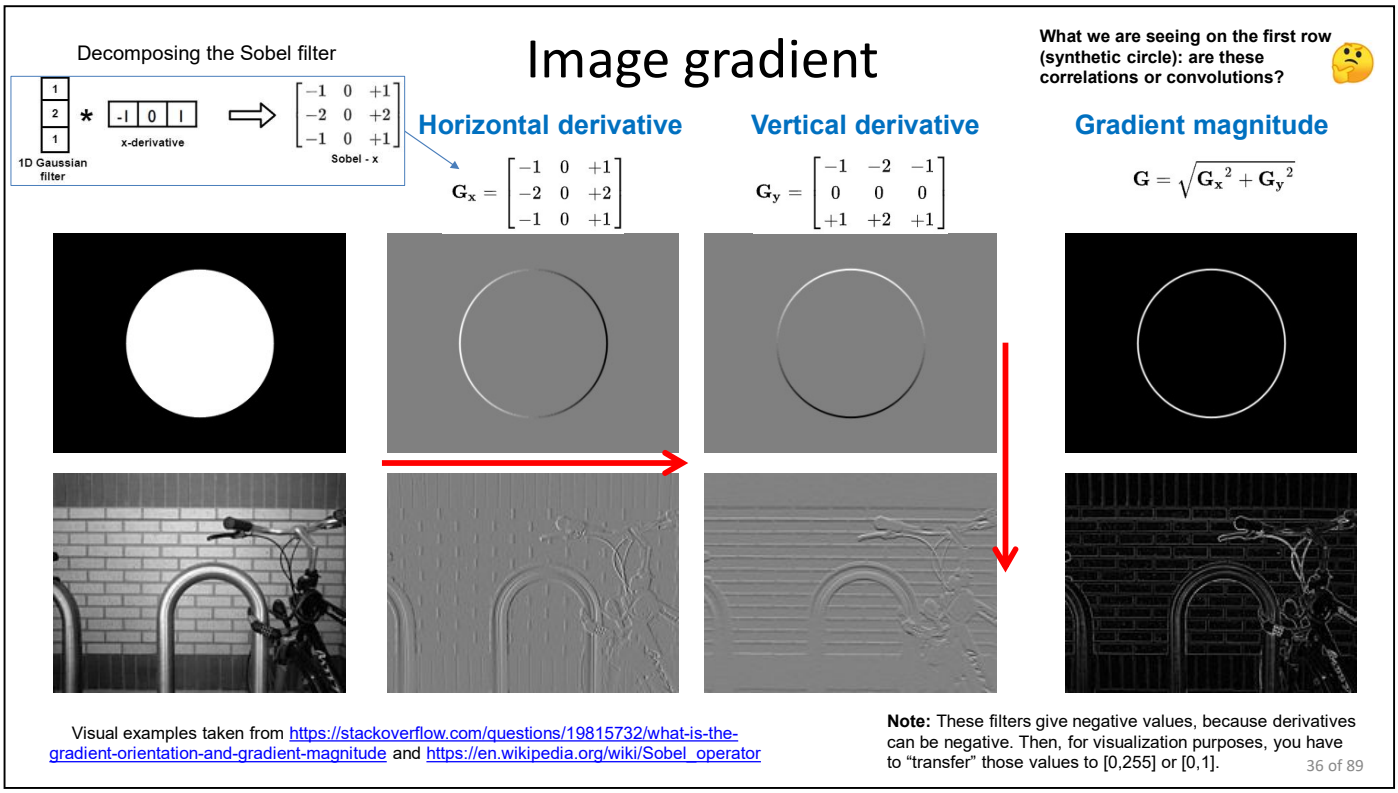
Source: Steve Seitz

35 of 89

La derivada de una imagen está íntimamente relacionada con el concepto de gradiente. El gradiente de una imagen está formado por sus dos derivadas parciales: una respecto a x y otra respecto a y.

Recordemos que el gradiente apunta en la dirección en la que la función aumenta más rápidamente. En nuestro caso, la función es la imagen f(x,y), que para cada posición o coordenada de píxel nos devuelve un valor de intensidad. Por tanto, el gradiente apunta en la dirección de máximo incremento de la intensidad.

El gradiente de una imagen no deja de ser un gradiente como cualquier otro y, por tanto, tiene una magnitud y una dirección. La magnitud nos indica cuánto cambia la intensidad y la dirección nos indica hacia dónde se produce ese incremento máximo.



En esta diapositiva vemos una aplicación del filtro de Sobel, uno de los operadores clásicos para aproximar las derivadas de una imagen. El operador de Sobel puede entenderse como la combinación de un kernel de derivación 1D con un pequeño kernel binomial de suavizado [1, 2, 1]. Mediante su producto exterior obtenemos los kernels 2D. Tenemos un kernel para aproximar la derivada en x, que responde especialmente a bordes verticales, y otro para aproximar la derivada en y, que responde especialmente a bordes horizontales.

En el centro de la diapositiva vemos dos ejemplos. El primero corresponde a una imagen sintética formada por un círculo blanco sobre fondo negro y el segundo a una fotografía real de una bicicleta delante de una pared. Vamos a centrarnos en el ejemplo sintético, ya que los principios son exactamente los mismos.

Si utilizamos el kernel de Sobel en x, [-1 0 1; -2 0 2; -1 0 1], lo desplazamos por toda la imagen calculando diferencias de intensidad en la dirección horizontal. En las regiones constantes, tanto negras como blancas, la respuesta es cero porque no existe variación de intensidad. En cambio, aparecen valores positivos y negativos en los dos lados del círculo, donde se producen las transiciones de negro a blanco y de blanco a negro.

Para visualizar la imagen de derivadas tenemos que representar también valores negativos. En esta diapositiva se ha utilizado una representación en la que los valores más negativos aparecen en negro, los más positivos en blanco y los valores próximos a cero en gris. Por eso las regiones constantes del círculo y del fondo aparecen grises.

Es importante tener en cuenta que las imágenes mostradas corresponden realmente a correlaciones, es decir, estamos aplicando los kernels tal y como aparecen escritos, sin invertirlos. Si realizásemos estrictamente una convolución, el kernel se invertiría y cambiaría el signo de la derivada. Por tanto, las regiones blancas y negras correspondientes a derivadas positivas y negativas aparecerían intercambiadas, mientras que las zonas próximas a cero seguirían apareciendo en gris.

Con el kernel de Sobel en y ocurre lo mismo, pero ahora medimos los cambios en la dirección vertical. Con la convención utilizada aquí, la parte superior del círculo produce una respuesta positiva y aparece blanca, mientras que la parte inferior produce una respuesta negativa y aparece negra.

Finalmente, podemos combinar ambas derivadas para calcular la magnitud del gradiente. Para cada píxel tomamos su derivada en x y su derivada en y, elevamos ambas al cuadrado, las sumamos y calculamos la raíz cuadrada. De esta forma obtenemos una medida no negativa de la intensidad del cambio, independientemente de la dirección en la que se produzca el borde.

Image gradient

Is there any difference if we apply cross-correlation or convolution with this kernel?? 🤔

Horizontal derivative

$$\mathbf{G_x} = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}$$

Vertical derivative

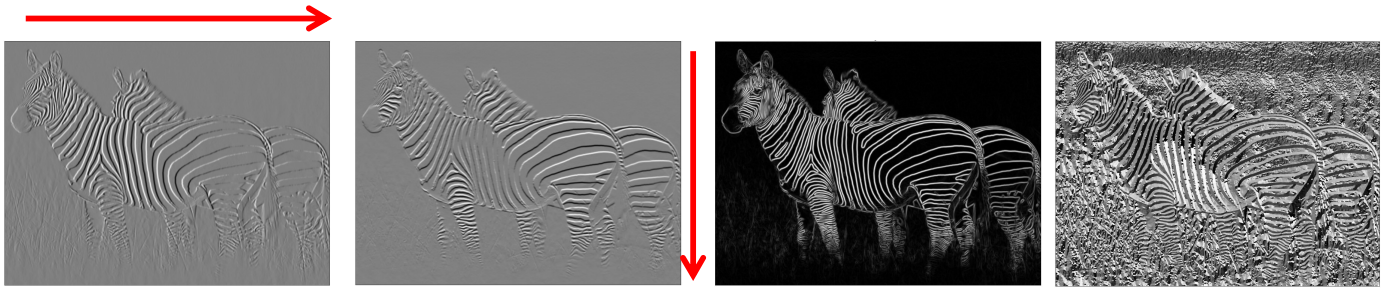
$$\mathbf{G_y} = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

Gradient magnitude

$$\mathbf{G} = \sqrt{\mathbf{G_x}^2 + \mathbf{G_y}^2}$$

Gradient orientation

$$\mathbf{\Theta} = \text{atan2}(\mathbf{G}_y, \mathbf{G}_x)$$



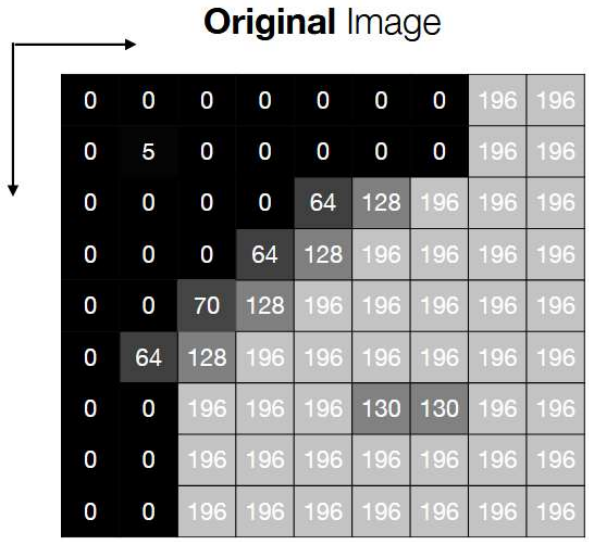
37 of 89

Ahora realizamos la misma operación, pero sobre una imagen real de dos cebras. Por un lado calculamos la derivada en x y, por otro, la derivada en y. A partir de ambas componentes podemos calcular la magnitud del gradiente, que ya no contiene valores negativos, y también su orientación en cada píxel, que podemos obtener mediante la arcotangente de la componente en y respecto a la componente en x.

¿Existe alguna diferencia entre aplicar correlación cruzada o convolución con estos kernels de Sobel? En este caso, la diferencia fundamental es el signo. Al pasar de correlación a convolución, el kernel se gira 180 grados y, como los kernels de Sobel son antisimétricos, esto equivale a cambiarles el signo. Por tanto, las derivadas en x e y cambian de signo.

La magnitud del gradiente, sin embargo, permanece exactamente igual, porque depende de los cuadrados de ambas componentes. La orientación cambia en 180 grados, es decir, el vector gradiente apunta en la dirección opuesta.

Detailed example of gradient calculation

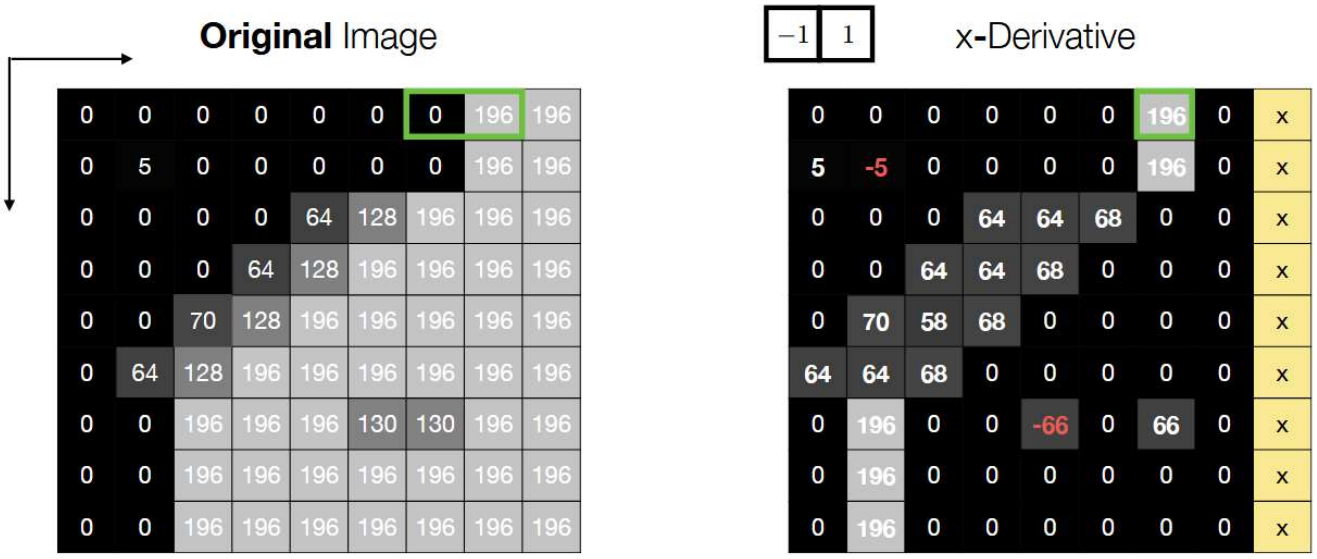


Source: L. Sigal (https://www.cs.ubc.ca/~lsigal/425_2022W2/Lecture10.pdf)

38 of 89

Y ahora, en aras de la completitud y la claridad, vamos a ver un ejemplo detallado de cómo calcular el gradiente de una imagen. En esta diapositiva partimos de una imagen de 9×9 píxeles en la que se muestran simultáneamente las distintas tonalidades de gris y, sobre cada píxel, su valor numérico de intensidad entre 0 y 255.

Detailed example of gradient calculation



Source: L. Sigal (https://www.cs.ubc.ca/~lsigal/425_2022W2/Lecture10.pdf) 39 of 89

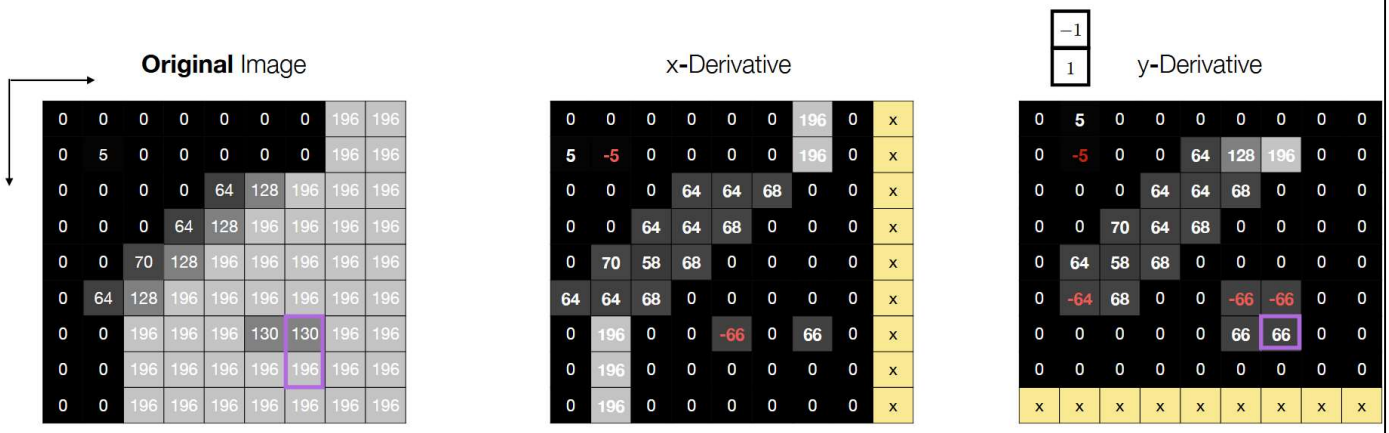
Vamos a utilizar un sencillo kernel de diferencias de tamaño 2 y comenzaremos calculando la derivada en x. Nuestro kernel es el vector fila [-1, 1], de manera que vamos a calcular diferencias entre píxeles vecinos en la dirección horizontal. En este caso asociamos el resultado con el píxel situado a la izquierda de la pareja.

Por ejemplo, en el recuadro verde que aparece en la imagen calculamos -1 por 0 más 1 por 196, obteniendo 196. Podemos hablar aquí de correlación, ya que estamos aplicando el kernel tal y como está escrito, sin invertirlo. Si desplazamos el kernel una posición hacia la derecha, obtenemos -196 + 196 = 0.

Cuando llegamos a la última columna ya no disponemos de un píxel a su derecha. Como en este ejemplo no estamos utilizando padding, esa operación simplemente no se calcula. Por eso los elementos de la última columna aparecen marcados en amarillo y con una cruz.

Como consecuencia, al aplicar un kernel horizontal de tamaño 2 sin padding, pasamos de una imagen de 9×9 a una imagen de derivadas en x de 9 filas y 8 columnas.

Detailed example of gradient calculation



Source: L. Sigal (https://www.cs.ubc.ca/~lsigal/425_2022W2/Lecture10.pdf)

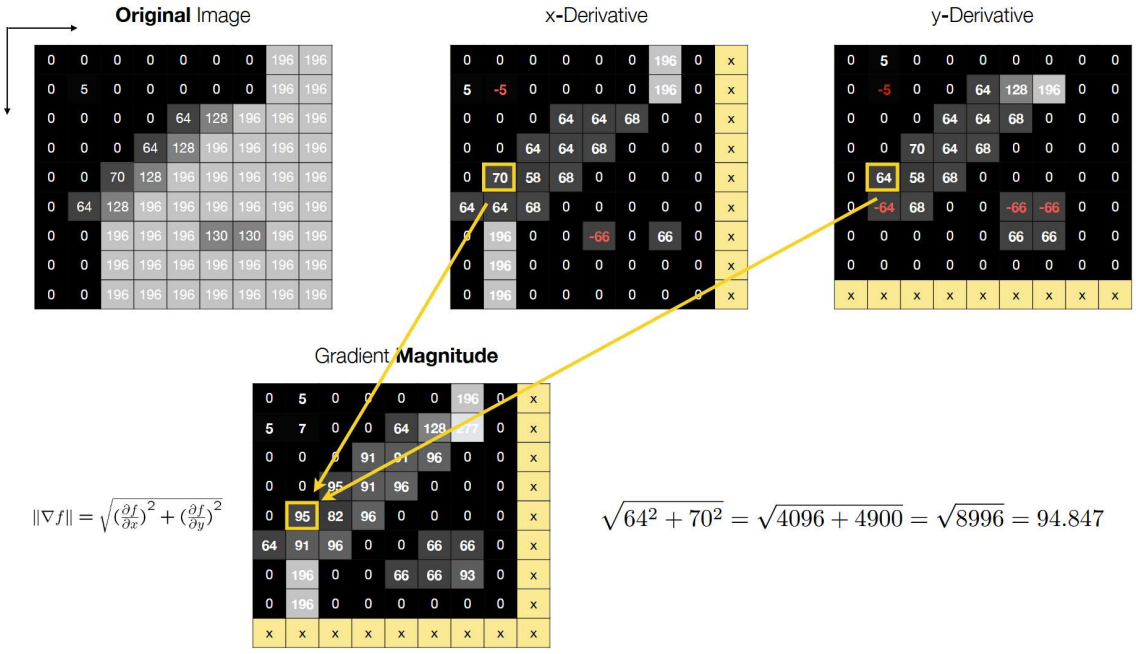
40 of 89

En esta diapositiva hacemos exactamente lo mismo, pero ahora calculamos también la derivada en y utilizando el kernel columna [-1, 1]. En este caso calculamos diferencias entre píxeles vecinos en la dirección vertical.

Como seguimos trabajando sin padding y asociamos el resultado con el píxel superior de cada pareja, al llegar a la última fila ya no disponemos de un píxel situado por debajo con el que calcular la diferencia. Por tanto, esa última fila no se calcula.

Así, partiendo de la imagen original de 9×9, la derivada en y tendrá 8 filas y 9 columnas.

Detailed example of gradient calculation



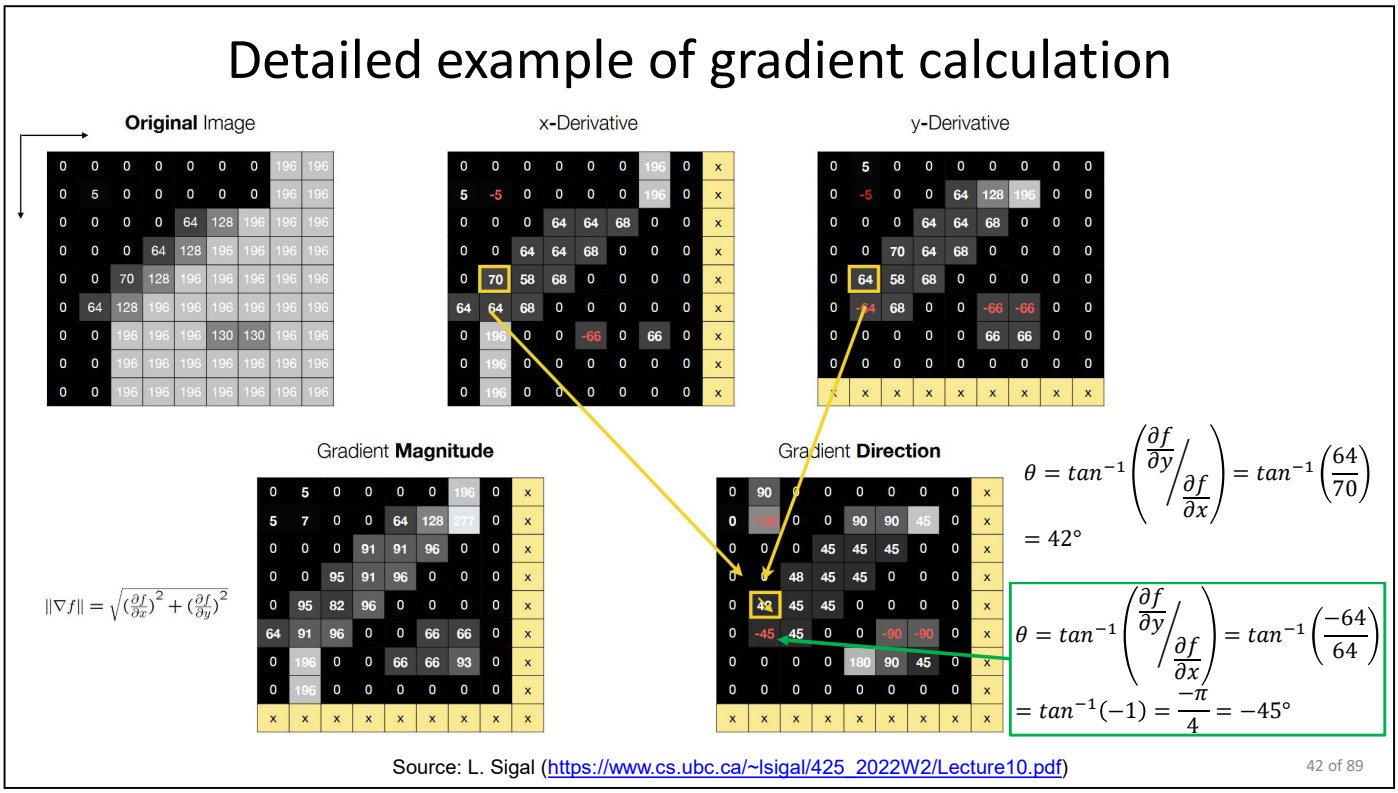
Source: L. Sigal (https://www.cs.ubc.ca/~lsigal/425_2022W2/Lecture10.pdf)

41 of 89

Ahora en esta diapositiva introducimos de forma explícita el concepto de magnitud del gradiente. Por ejemplo, para la posición marcada en amarillo tomamos el valor de la derivada en x, que es 70, y el valor de la derivada en y en ese mismo píxel, que es 64.

La magnitud del gradiente se calcula elevando ambos valores al cuadrado, sumándolos y calculando la raíz cuadrada. En este caso obtenemos raíz cuadrada de $70^2 + 64^2$, lo que da aproximadamente 94,847.

Este valor nos indica la intensidad del cambio local en ese punto de la imagen, independientemente de la dirección en la que se esté produciendo.



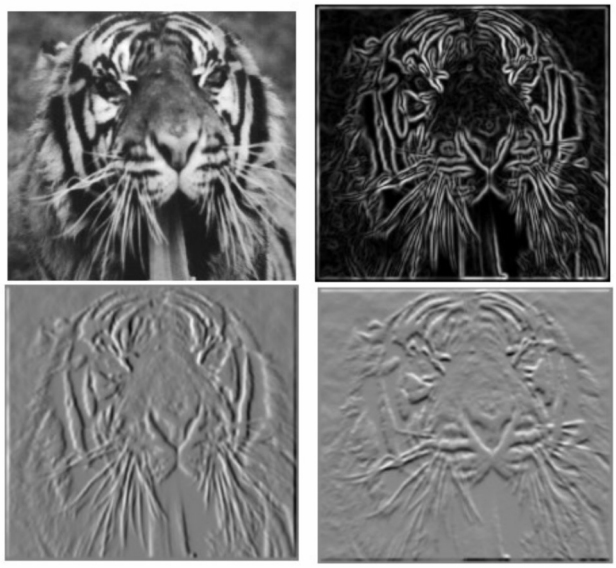
Y ahora hacemos exactamente lo mismo para calcular la dirección del gradiente. Para el píxel marcado tomamos el valor de la derivada en y, que es 64, y el valor de la derivada en x, que es 70. Calculamos 64 dividido entre 70 y aplicamos la arcotangente. El resultado es un ángulo de aproximadamente 42 grados.

Si ahora consideramos el píxel justo debajo del anterior, la derivada en y vale -64 y la derivada en x vale 64. Por tanto, tenemos -64 dividido entre 64, es decir, -1. La arcotangente de -1 es -pi/4, que corresponde a -45 grados. Este es precisamente el valor al que apunta la flecha verde en la diapositiva.

En este ejemplo se ha optado por no calcular la dirección del gradiente cuando falta alguna de sus dos componentes, como ocurre en la última columna para la derivada en x o en la última fila para la derivada en y.

Aquí estamos utilizando la arcotangente del cociente entre Gy y Gx, pero en la práctica suele ser preferible utilizar atan2(Gy, Gx). La función atan2 tiene en cuenta por separado los signos de Gy y Gx, por lo que puede determinar correctamente el cuadrante en el que se encuentra el vector gradiente y devolver una dirección a lo largo de los 360 grados. En cambio, la arcotangente convencional recibe únicamente el cociente Gy/Gx, pierde información sobre los signos individuales de ambas componentes y devuelve valores en un intervalo de 180 grados. Por ello puede existir una ambigüedad de 180 grados en la dirección del gradiente (es decir, no puede distinguir vectores que apuntan en direcciones opuestas).

Image gradient



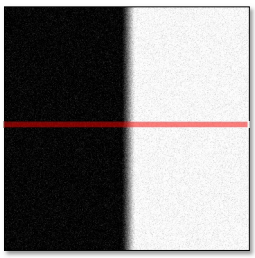
🤔

Which one represents the gradient magnitude?

Which one shows changes with respect to x?

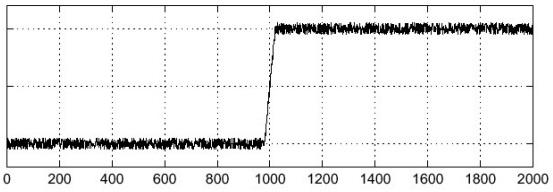
Which one shows changes with respect to y?

Effects of noise

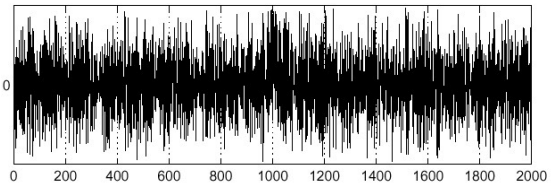


Noisy input image

$f(x)$



$\frac{d}{dx}f(x)$



Derivatives amplify noise (high-frequency components)!

Where is the edge?

Source: S. Seitz

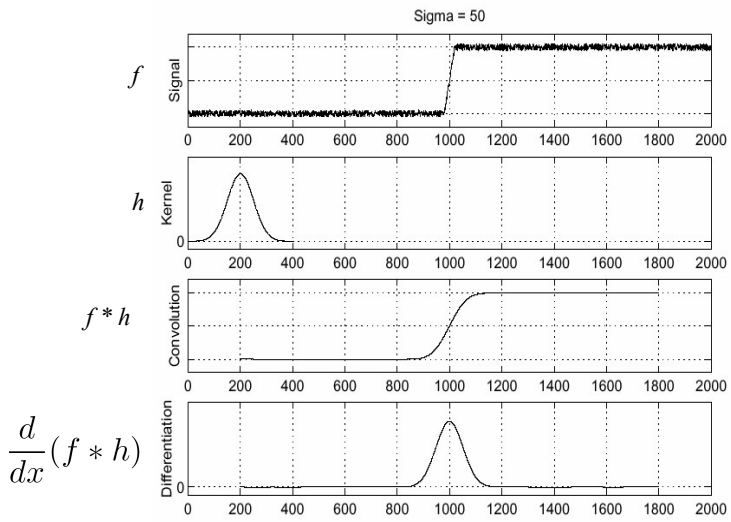
44 of 89

Ahora vamos a ver de qué manera el ruido puede afectar al cálculo de derivadas. Imaginemos una imagen ruidosa como la que aparece en el centro-izquierda de la diapositiva: la mitad izquierda es oscura y la mitad derecha es clara, pero hemos añadido ruido gaussiano.

Si observamos el perfil de intensidades a lo largo de la línea roja horizontal, obtenemos una señal como f(x), en la que tenemos valores próximos a negro con pequeñas fluctuaciones, una transición hacia la región clara y, finalmente, valores próximos a blanco que también presentan ruido.

El problema es que la operación de derivación es especialmente sensible al ruido. Las pequeñas variaciones de intensidad introducidas por el ruido producen también respuestas importantes en la derivada. Como consecuencia, al derivar directamente la señal obtenemos una respuesta ruidosa, en la que puede resultar difícil distinguir el borde verdadero de las variaciones producidas simplemente por el ruido.

Solution: smooth first



How many convolutions do we have here?? 🤔

To find edges, look for peaks $\frac{d}{dx}(f * h)$

Source: S. Seitz

45 of 89

Y precisamente en esta diapositiva vemos la lógica de por qué, cuando queremos detectar bordes mediante filtros de derivadas, suele ser conveniente suavizar primero la señal o la imagen.

Partimos de una señal ruidosa f. En primer lugar, la convolucionamos con un kernel de suavizado h, obteniendo una versión suavizada de la señal. De esta forma reducimos las pequeñas variaciones de alta frecuencia producidas por el ruido.

A continuación, calculamos la derivada de esa señal suavizada. En la gráfica final vemos que la respuesta de la derivada es mucho más limpia y aparece un máximo claramente localizado alrededor de x = 1000, que corresponde precisamente a la posición del borde.

Por tanto, una manera de detectar bordes consiste en buscar extremos (picos y valles pronunciados) en la derivada de la señal suavizada. Conceptualmente estamos realizando dos operaciones sucesivas: primero una convolución de suavizado y después una convolución con un filtro de derivadas.

Associative property of convolution

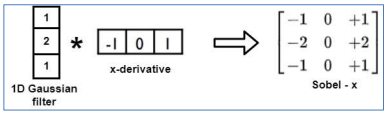
- Differentiation is convolution, and convolution is associative:

$$\frac{d}{dx}(f * h) = f * \frac{d}{dx}h$$

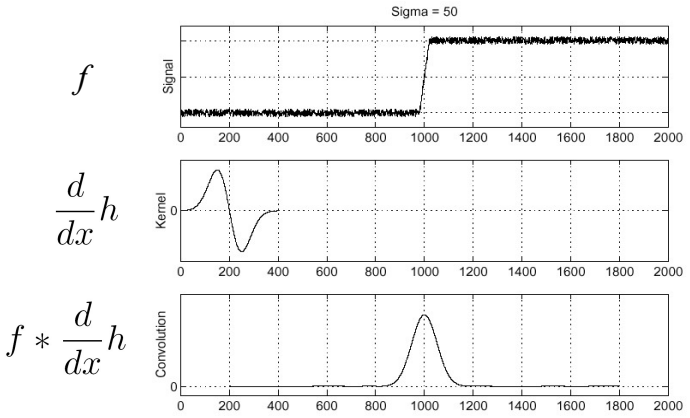
- This saves us one operation:

Computing derivatives many times implies smoothing (in one direction) and differentiating (in the other)

Examples:



$$\frac{1}{2}\begin{bmatrix}1 & 1\end{bmatrix} * \begin{bmatrix}1 & -1\end{bmatrix} = \frac{1}{2}\begin{bmatrix}1 & 0 & -1\end{bmatrix}$$



Source: S. Seitz

46 of 89

Y aquí podemos combinar varias de las ideas que hemos visto anteriormente. Si queremos calcular la derivada de una señal previamente suavizada, partimos de la convolución de la señal f con un kernel de suavizado h y posteriormente calculamos la derivada del resultado.

Una propiedad fundamental nos permite escribir la derivada de la convolución de f con h como la convolución de f con la derivada de h. Por tanto, en lugar de realizar primero una convolución de suavizado y después una operación de derivación, podemos construir directamente un kernel que sea la derivada del kernel de suavizado y convolucionar la señal una única vez con él.

Si h es una gaussiana, obtenemos precisamente un kernel de primera derivada de la gaussiana. Al convolucionar directamente la señal con este kernel obtenemos el mismo resultado que si primero hubiésemos suavizado con la gaussiana y después hubiésemos calculado la derivada.

En dos dimensiones aparece además una idea especialmente interesante gracias a la separabilidad de la gaussiana. Para calcular, por ejemplo, la derivada respecto a x, podemos derivar en la dirección x y suavizar en la dirección y. Esta misma filosofía aparece en el operador de Sobel: el kernel de Sobel en x combina un filtro de diferencias en la

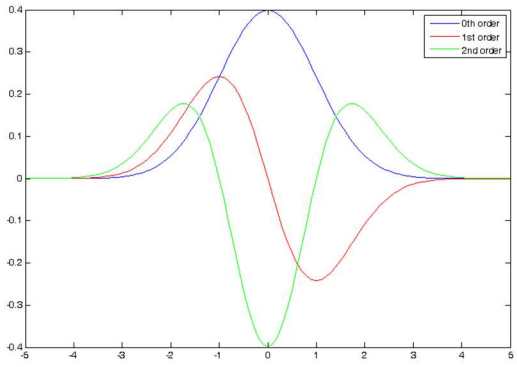
dirección x con el kernel binomial [1, 2, 1] de suavizado en la dirección y.

The 1D Gaussian and its derivatives

$$g_\sigma(x) = \frac{1}{\sqrt{2\pi}\sigma}\exp\left(-\frac{x^2}{2\sigma^2}\right)$$

$$g'_\sigma(x) = \frac{\partial g_\sigma(x)}{\partial x} = -\frac{x}{\sigma^2}\frac{1}{\sqrt{2\pi}\sigma}\exp\left(-\frac{x^2}{2\sigma^2}\right) = -\frac{x}{\sigma^2}g_\sigma(x)$$

$$g''_\sigma(x) = \frac{\partial^2 g_\sigma(x)}{\partial x^2} = \left(\frac{x^2}{\sigma^4} - \frac{1}{\sigma^2}\right)\frac{1}{\sqrt{2\pi}\sigma}\exp\left(-\frac{x^2}{2\sigma^2}\right) = \left(\frac{x^2}{\sigma^4} - \frac{1}{\sigma^2}\right)g_\sigma(x)$$

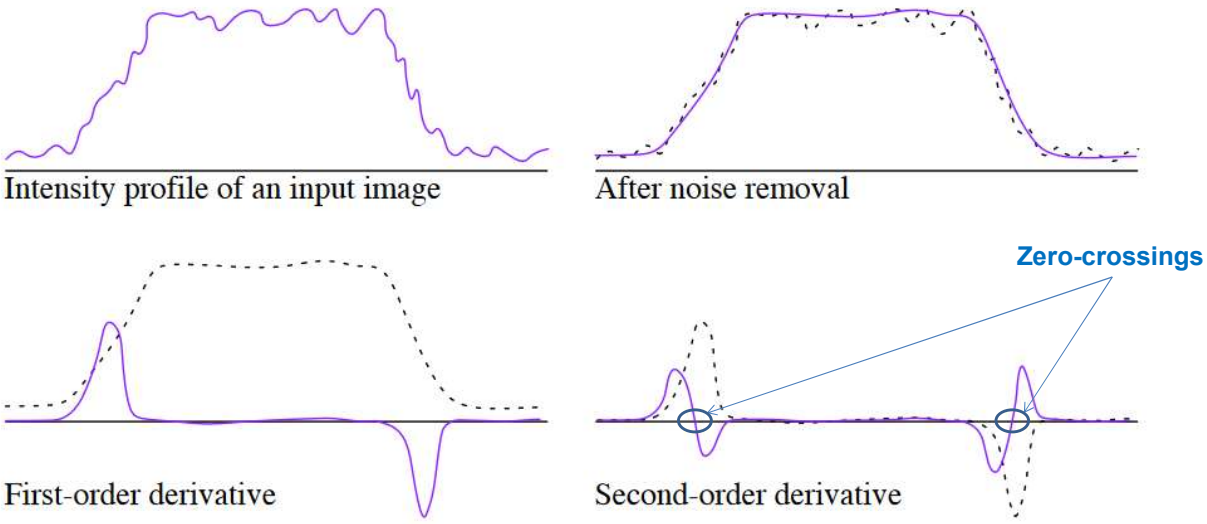


En esta diapositiva simplemente visualizamos la gaussiana unidimensional y sus dos primeras derivadas. En este caso se incluye también la constante de normalización de la gaussiana.

La curva azul representa la derivada de orden cero, es decir, la propia función gaussiana. La curva roja representa su primera derivada y la curva verde su segunda derivada.

Esta representación nos permite observar cómo cambia la forma de la función a medida que aumentamos el orden de derivación y nos servirá para relacionar posteriormente estas funciones con distintos tipos de filtros de derivadas.

Edge detection using first- and second-order derivatives



Main advantage of second derivatives?
They provide a **convenient way to localize edges through zero-crossings**. But be careful with noise!
Zero-crossings provide a **simple criterion for edge localization** (see https://www.cs.toronto.edu/~mangas/teaching/320/slides/CSC320L05.pdf)

https://www.cs.auckland.ac.nz/~rklette/TeachAuckland.html/775/pdfs/D06-LocalOp1.pdf

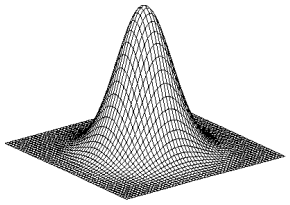
Esta diapositiva es especialmente importante para entender cómo se comportan las derivadas en presencia de bordes. Arriba a la izquierda tenemos el perfil de intensidad de una señal de entrada ruidosa. Después de aplicar un suavizado, obtenemos un perfil más regular, como el que aparece a continuación.

Si calculamos la primera derivada de ese perfil suavizado, vemos que aparece un máximo positivo en la transición ascendente, asociado al paso de intensidades bajas a intensidades altas. En la zona de la meseta la derivada permanece próxima a cero, porque apenas hay variación. Finalmente, en la transición descendente aparece un mínimo, es decir, una respuesta negativa asociada al paso de intensidades altas a intensidades bajas.

Si derivamos una segunda vez, obtenemos la segunda derivada. En este caso, los bordes pueden localizarse buscando cruces por cero de la segunda derivada. La lógica es que los máximos y mínimos de la primera derivada se corresponden con puntos en los que su derivada, es decir, la segunda derivada, cambia de signo y pasa por cero.

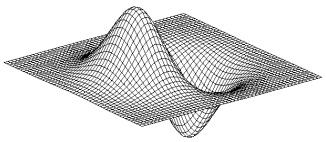
Por tanto, con la primera derivada buscamos extremos de gran magnitud, mientras que con la segunda derivada buscamos zero crossings. En ambos casos debemos tener cuidado con respuestas espurias producidas por ruido u otras variaciones de la señal.

2D edge detection filters



Gaussian

$$h_\sigma(u,v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$



derivative of Gaussian (*x*)

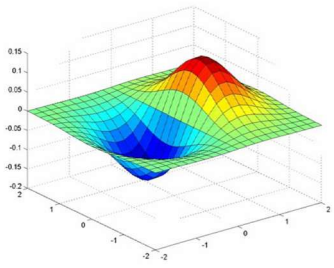
$$\frac{\partial}{\partial x} h_\sigma(u,v)$$

49 of 89

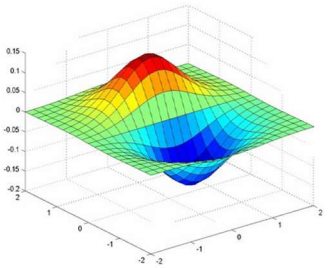
Y en 2D se mantienen las mismas ideas que hemos comentado hasta ahora. En esta diapositiva vemos a la izquierda la gaussiana bidimensional, con su forma de campana característica, y a la derecha su primera derivada respecto a una de las direcciones, por ejemplo x.

La forma que observamos es, esencialmente, la extensión bidimensional de la primera derivada que vimos en 1D: aparece una región positiva a un lado, una región negativa al otro y un cruce por cero en el centro. La diferencia es que ahora esa estructura se extiende a lo largo de la segunda dimensión.

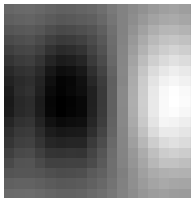
Derivative of Gaussian filter

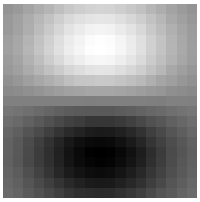


***x*-direction**



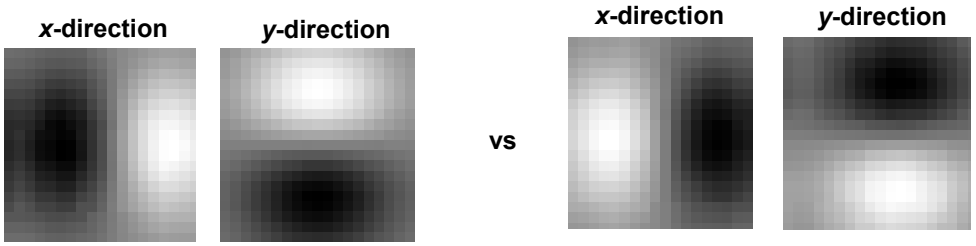
***y*-direction**





Derivative of Gaussian filter

Is the sign really important?? 🤔



Common 2D Gaussian derivative kernels

Prewitt

$$Prewitt_x = \frac{1}{3}\begin{bmatrix}1\\1\\1\end{bmatrix} * \frac{1}{2}\begin{bmatrix}1 & 0 & -1\end{bmatrix} = \frac{1}{6}\begin{bmatrix}1 & 0 & -1\\1 & 0 & -1\\1 & 0 & -1\end{bmatrix}$$

$$Prewitt_y = \frac{1}{3}\begin{bmatrix}1 & 1 & 1\end{bmatrix} * \frac{1}{2}\begin{bmatrix}1\\0\\-1\end{bmatrix} = \frac{1}{6}\begin{bmatrix}1 & 1 & 1\\0 & 0 & 0\\-1 & -1 & -1\end{bmatrix}$$

Sobel

$$Sobel_x = \frac{1}{4}\begin{bmatrix}1\\2\\1\end{bmatrix} * \frac{1}{2}\begin{bmatrix}1 & 0 & -1\end{bmatrix} = \frac{1}{8}\begin{bmatrix}1 & 0 & -1\\2 & 0 & -2\\1 & 0 & -1\end{bmatrix}$$

$$Sobel_y = \frac{1}{4}\begin{bmatrix}1 & 2 & 1\end{bmatrix} * \frac{1}{2}\begin{bmatrix}1\\0\\-1\end{bmatrix} = \frac{1}{8}\begin{bmatrix}1 & 2 & 1\\0 & 0 & 0\\-1 & -2 & -1\end{bmatrix}$$

Scharr

$$Scharr_x = \frac{1}{16}\begin{bmatrix}3\\10\\3\end{bmatrix} * \frac{1}{2}\begin{bmatrix}1 & 0 & -1\end{bmatrix} = \frac{1}{32}\begin{bmatrix}3 & 0 & -3\\10 & 0 & -10\\3 & 0 & -3\end{bmatrix}$$

$$Scharr_y = \frac{1}{16}\begin{bmatrix}3 & 10 & 3\end{bmatrix} * \frac{1}{2}\begin{bmatrix}1\\0\\-1\end{bmatrix} = \frac{1}{32}\begin{bmatrix}3 & 10 & 3\\0 & 0 & 0\\-3 & -10 & -3\end{bmatrix}$$

The standard definition of the Sobel operator omits the 1/8 term

- **doesn't make a difference for edge detection**

S. Birchfield, Clemson Univ., ECE 847, http://www.ces.clemson.edu/~stb/ece847

52 of 89

En esta diapositiva se muestran algunos de los kernels clásicos para aproximar derivadas en imágenes 2D, como Prewitt, Sobel o Scharr.

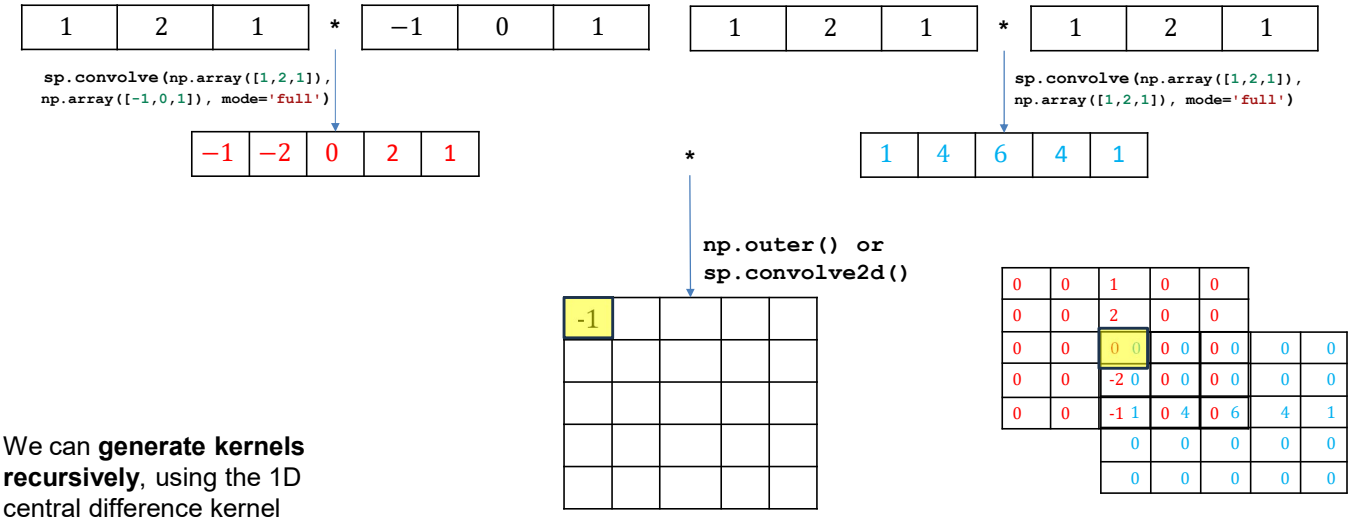
Por ejemplo, el operador de Prewitt en x puede entenderse como la combinación de un filtro de suavizado vertical [1, 1, 1]/3 con un filtro de diferencias centrales horizontal [1, 0, -1]/2. El producto exterior de ambos da lugar al kernel 2D correspondiente, con un factor global de 1/6. Para la derivada en y se utiliza la misma idea, intercambiando las direcciones.

Con Sobel ocurre algo muy parecido, aunque el suavizado utiliza el kernel binomial [1, 2, 1]. En ese caso aparece un factor global de 1/8. Scharr utiliza otra combinación de coeficientes.

Puede llamar la atención que aquí mostremos explícitamente esos factores constantes delante de los kernels. En muchas definiciones prácticas, por ejemplo, del operador de Sobel, el factor 1/8 se omite. Esto no cambia la localización de los bordes: simplemente escala todos los valores de la derivada por una constante. Por eso, en la mayor parte de tareas de detección de bordes, donde lo importante es la respuesta relativa o superar un cierto umbral, ese factor se ignora.

Creating larger derivative kernels

If we want a 5x5 Sobel kernel:



We can **generate kernels recursively**, using the 1D central difference kernel and the binomial!

53 of 89

Hasta ahora hemos trabajado principalmente con kernels de tamaño 3, tanto en 1D como en 2D. Pero también podemos construir operadores de Sobel de mayor tamaño. Por ejemplo, imaginemos que queremos obtener un kernel de Sobel de 5×5.

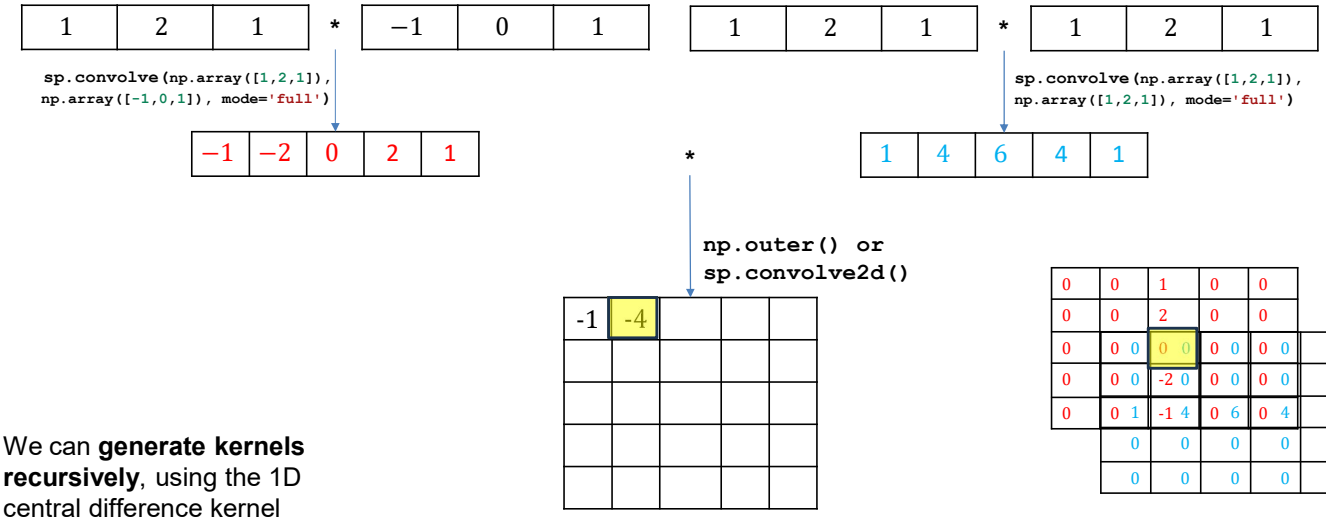
Podemos partir de los kernels 1D de tamaño 3. Si convolucionamos el kernel binomial [1, 2, 1] con el kernel de diferencias centrales [-1, 0, 1], obtenemos [-1, -2, 0, 2, 1], que será nuestro kernel 1D de derivación de tamaño 5. Por otro lado, si convolucionamos [1, 2, 1] consigo mismo, obtenemos [1, 4, 6, 4, 1], que proporciona el componente 1D de suavizado de tamaño 5.

Una vez tenemos estos dos vectores de tamaño 5, podemos obtener el kernel Sobel 2D de 5×5 mediante su producto exterior: utilizamos el kernel de derivación en una dirección y el kernel de suavizado en la dirección perpendicular. Equivalentemente, podríamos interpretar esta operación como la convolución 2D de los correspondientes kernels 1D embebidos en dos dimensiones.

Esta construcción puede extenderse recursivamente para obtener kernels Sobel de tamaños mayores. La idea consiste en ir ampliando tanto el componente de derivación como el de suavizado mediante convoluciones con el kernel binomial [1, 2, 1].

Creating larger derivative kernels

If we want a 5x5 Sobel kernel:

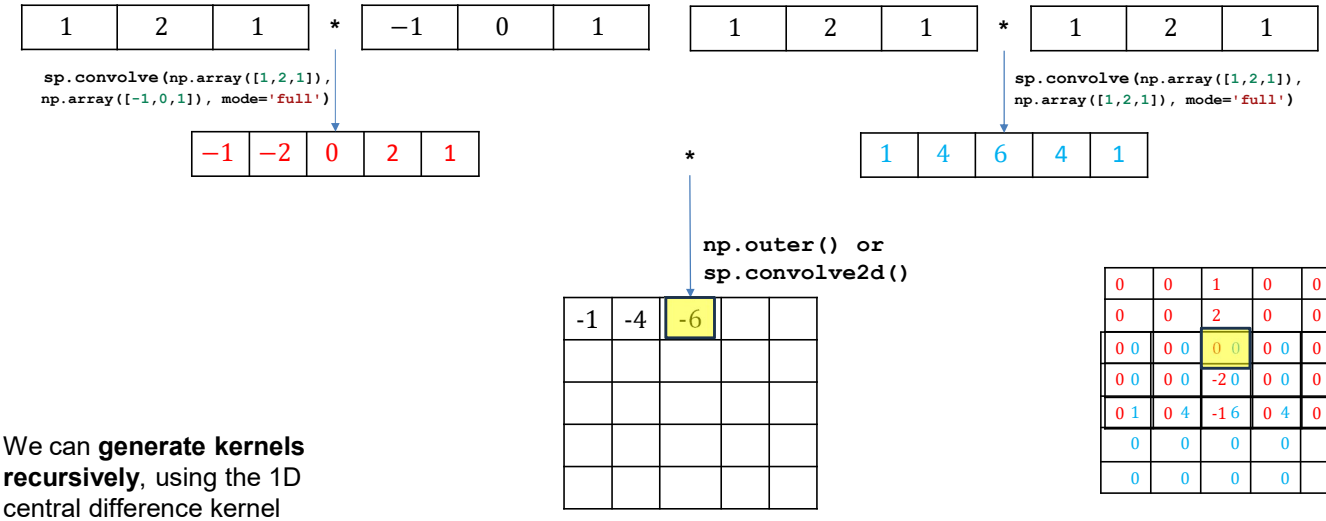


We can **generate kernels recursively**, using the 1D central difference kernel and the binomial!

54 of 89

Creating larger derivative kernels

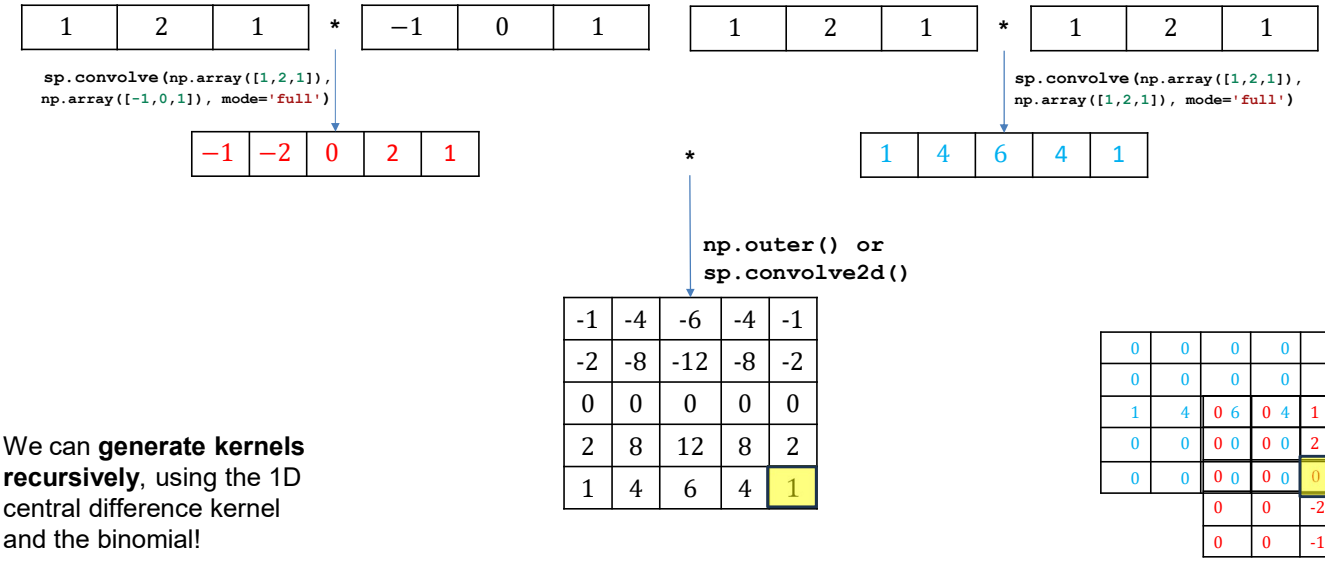
If we want a 5x5 Sobel kernel:



We can **generate kernels recursively**, using the 1D central difference kernel and the binomial!

Creating larger derivative kernels

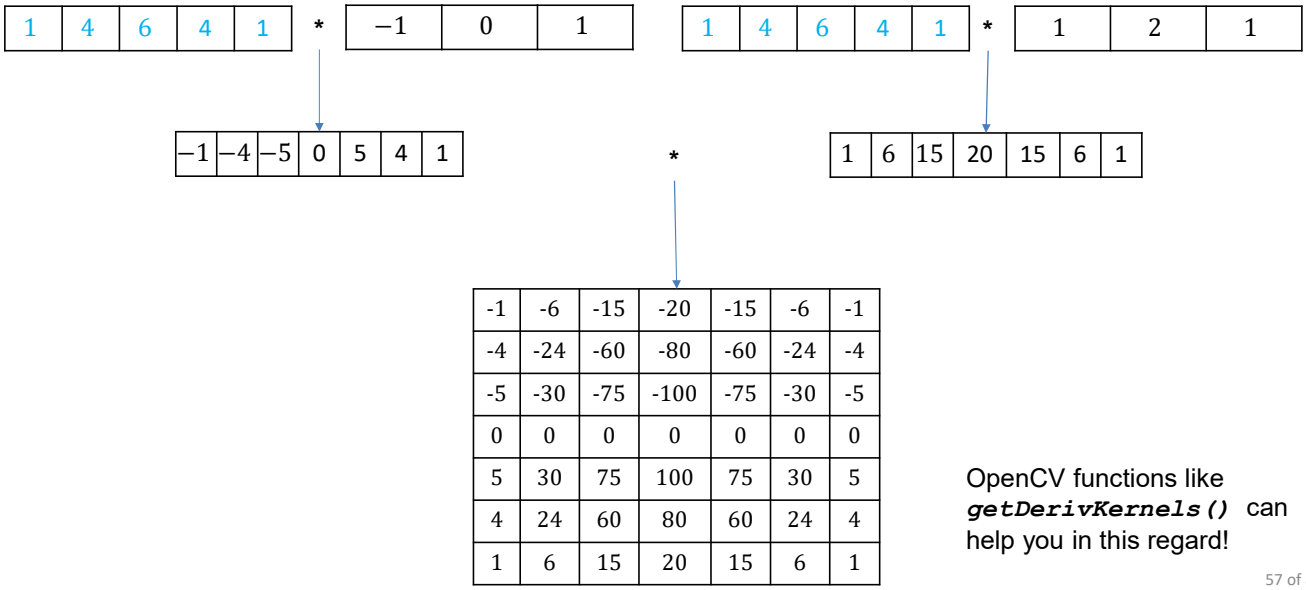
If we want a 5x5 Sobel kernel:



We can **generate kernels recursively**, using the 1D central difference kernel and the binomial!

Creating larger derivative kernels

If we want a 7x7 Sobel kernel:



-1	-6	-15	-20	-15	-6	-1
-4	-24	-60	-80	-60	-24	-4
-5	-30	-75	-100	-75	-30	-5
0	0	0	0	0	0	0
5	30	75	100	75	30	5
4	24	60	80	60	24	4
1	6	15	20	15	6	1

OpenCV functions like ***`getDerivKernels()`*** can help you in this regard!

57 of 89

Y lo mismo podemos hacer si queremos construir un kernel de Sobel de tamaño 7×7. Partimos del kernel binomial de tamaño 5, [1, 4, 6, 4, 1], y lo convolucionamos con el kernel de diferencias [-1, 0, 1]. De esta forma obtenemos el kernel 1D de derivación de tamaño 7: [-1, -4, -5, 0, 5, 4, 1].

Por otro lado, convolucionamos el kernel binomial de tamaño 5 con el kernel binomial de tamaño 3, [1, 2, 1], obteniendo el kernel de suavizado de tamaño 7: [1, 6, 15, 20, 15, 6, 1].

Ya tenemos, por tanto, dos kernels 1D de tamaño 7: uno de derivación y otro de suavizado. Mediante el producto exterior de ambos podemos construir el correspondiente kernel de Sobel 2D de tamaño 7×7, aplicando la derivación en una dirección y el suavizado en la perpendicular.

Esta misma idea es la que subyace a funciones de OpenCV como getDerivKernels, que permite generar los kernels 1D utilizados por Sobel para distintos tamaños.

Separability

First derivative of 2D Gaussian is separable:

$$
\begin{aligned}
G(x,y) &= \frac{1}{2\pi\sigma^2}\exp\left(\frac{-(x^2+y^2)}{2\sigma^2}\right) \\
&= \frac{1}{\sqrt{2\pi\sigma^2}}\exp\left(-\frac{x^2}{2\sigma^2}\right)\cdot\frac{1}{\sqrt{2\pi\sigma^2}}\exp\left(-\frac{y^2}{2\sigma^2}\right) \\
&= G_h(x)\cdot G_v(y) \\
\frac{\partial G(x,y)}{\partial x} &= -\frac{x}{\sigma^2}\cdot\frac{1}{2\pi\sigma^2}\exp\left(-\frac{x^2}{2\sigma^2}\right)\exp\left(-\frac{y^2}{2\sigma^2}\right) \\
&= G'_h(x)\cdot G_v(y)
\end{aligned}
$$

horizontal 1D Gaussian derivative kernel

vertical 1D Gaussian kernel

smooth in one direction, differentiate in the other

This allows us to know **which 1D kernels to apply, and in what order, to calculate the derivatives of an image.** In the previous example, the first derivative on X.

58 of 89

Ahora vamos a detenernos con un poco más de calma en la separabilidad de las derivadas de la función gaussiana. Partimos de la gaussiana 2D, que puede expresarse como el producto de dos gaussianas 1D, una en x y otra en y. Es decir, G(x,y) = G(x)G(y).

Si ahora calculamos la derivada respecto a x, G(y) se comporta como una constante, ya que no depende de x. Por tanto, obtenemos G'(x)G(y). Esto nos dice que la derivada respecto a x de una gaussiana 2D también es separable: en la dirección horizontal tenemos un kernel 1D correspondiente a la derivada de la gaussiana, mientras que en la dirección vertical tenemos un kernel gaussiano 1D de suavizado.

En otras palabras, al calcular una derivada de gaussiana en x estamos combinando derivación en una dirección con suavizado en la dirección perpendicular. Para la derivada respecto a y ocurre exactamente lo mismo, intercambiando ambas direcciones.

Esta propiedad es especialmente útil desde el punto de vista computacional, porque no necesitamos aplicar directamente un kernel 2D: podemos aplicar sucesivamente los dos kernels 1D correspondientes.

Separability

First derivative of 2D Gaussian is separable:

$$
\begin{aligned}
G(x,y) &= \frac{1}{2\pi\sigma^2}\exp\left(\frac{-(x^2+y^2)}{2\sigma^2}\right) \\
&= \frac{1}{\sqrt{2\pi\sigma^2}}\exp\left(-\frac{x^2}{2\sigma^2}\right)\cdot\frac{1}{\sqrt{2\pi\sigma^2}}\exp\left(-\frac{y^2}{2\sigma^2}\right) \\
&= G_h(x)\cdot G_v(y) \\
\frac{\partial G(x,y)}{\partial x} &= -\frac{x}{\sigma^2}\cdot\frac{1}{2\pi\sigma^2}\exp\left(-\frac{x^2}{2\sigma^2}\right)\exp\left(-\frac{y^2}{2\sigma^2}\right) \\
&= G'_h(x)\cdot G_v(y)
\end{aligned}
$$

A 2D convolution with a separable kernel can be replaced by two consecutive 1D convolutions.

59 of 89

Si un kernel 2D es separable, la convolución 2D puede sustituirse por dos convoluciones 1D consecutivas: una en una dirección y otra en la dirección perpendicular.

En nuestro caso, para calcular la derivada en x de una gaussiana, podemos aplicar primero el kernel gaussiano 1D en y y después el kernel correspondiente a la derivada de la gaussiana en x, o al revés. El resultado es el mismo.

La principal ventaja es computacional: en lugar de realizar una convolución con un kernel de tamaño m×m, realizamos dos convoluciones con kernels de tamaño m, reduciendo considerablemente el número de operaciones.

Separability

1. We know that the 1st derivative of a 2D Gaussian is separable.

$$\begin{aligned}\frac{\partial G(x,y)}{\partial x} &= -\frac{x}{\sigma^2}\cdot\frac{1}{2\pi\sigma^2}\exp\left(-\frac{x^2}{2\sigma^2}\right)\exp\left(-\frac{y^2}{2\sigma^2}\right)\\ &= G'_h(x)\cdot G_v(y)\end{aligned}$$

The partial derivative of a Gaussian is composed of the derivative in one direction and the Gaussian in the other.

2. From a practical point of view, we are interested in implicitly **smoothing the signal to avoid magnifying the noise**. So, we smooth in one direction and differentiate in the other.

3. Mathematically, **a separable kernel function leads to a separable convolution**. See http://www.songho.ca/dsp/convolution/convolution2d_separable.html

$$\begin{aligned}y[m,n] = (h_1[m]\cdot h_2[n]) * x[m,n] &= h_2[n] * (h_1[m] * x[m,n])\\ &= h_1[m] * (h_2[n] * x[m,n])\end{aligned}$$

60 of 89

Y ahora viene una pregunta relevante. En la diapositiva anterior hemos visto que la gaussiana bidimensional es separable, es decir, que puede expresarse como el producto de dos gaussianas unidimensionales, una en x y otra en y. La pregunta ahora es: ¿cómo se relaciona ese producto con la convolución que aplicamos sobre una imagen?

Podemos verlo desde varios puntos de vista. En primer lugar, la derivada de una gaussiana 2D también es separable. Por ejemplo, la derivada respecto a x puede escribirse como el producto de una derivada de gaussiana 1D en x y una gaussiana 1D en y. Por tanto, aparecen de forma natural dos componentes diferentes: derivación en una dirección y suavizado en la otra.

En segundo lugar, esto tiene una interpretación práctica muy conveniente. Sabemos que las derivadas amplifican el ruido y, por ello, nos interesa combinar la diferenciación con un suavizado. Así, mientras calculamos la derivada en una dirección, suavizamos en la dirección perpendicular.

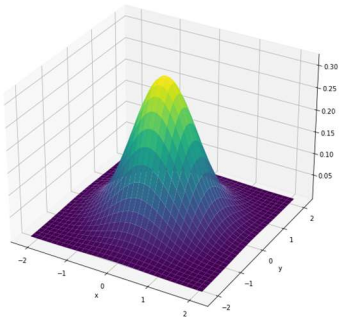
Finalmente, está la justificación matemática: si un kernel 2D es separable y puede escribirse como el producto de dos funciones 1D, entonces la convolución 2D de una imagen con ese kernel puede calcularse mediante dos convoluciones 1D sucesivas, una en cada dirección. Una demostración paso a paso de esta propiedad puede encontrarse aquí:

http://www.songho.ca/dsp/convolution/convolution2d_separable.html

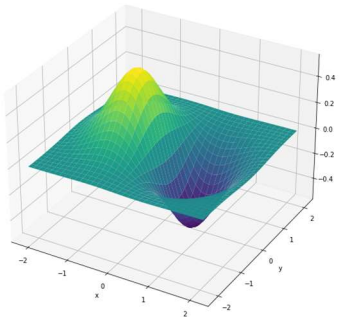
Laplacian of Gaussian

- How many 2nd derivative filters do we have? There are four 2nd partial derivative filters.
- In practice, it's handy to define a single 2nd derivative filter—the Laplacian
  - Instead of using $\frac{\partial^2(G_\sigma(x,y))}{\partial x^2} = \left(\frac{x^2}{\sigma^4} - \frac{1}{\sigma^2}\right) G_\sigma(x,y)$ and $\frac{\partial^2(G_\sigma(x,y))}{\partial y^2} = \left(\frac{y^2}{\sigma^4} - \frac{1}{\sigma^2}\right) G_\sigma(x,y)$
  - We employ the Laplacian of Gaussian (LoG) filter

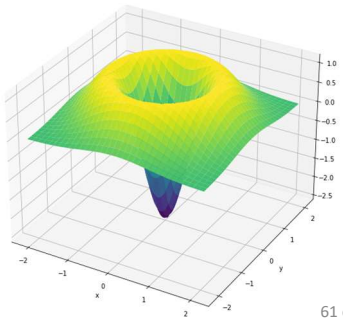
Gaussian



1st derivative of Gaussian (x)



LoG



61 of 89

Ahora nos adentramos en las derivadas de segundo orden. En principio podemos considerar tres derivadas segundas distintas: la segunda derivada respecto a x, la segunda derivada respecto a y, y la derivada mixta, primero respecto a x y después respecto a y. Bajo las condiciones habituales de suavidad, esta derivada mixta es igual si invertimos el orden, es decir, derivar primero respecto a y y después respecto a x. Por eso solemos hablar de tres derivadas segundas distintas y no de cuatro (aunque cuatro es lo que se indica explícitamente en la diapositiva).

En visión por computador resulta especialmente útil combinar las dos derivadas segundas puras mediante el operador laplaciano. En el caso de una gaussiana, obtenemos la Laplaciana de la Gaussiana, o LoG, que corresponde a sumar la segunda derivada de la gaussiana respecto a x y la segunda derivada respecto a y.

De esta forma obtenemos un único filtro de segundo orden, que además tiene la ventaja de ser isotrópico, es decir, no privilegia una dirección concreta. Su forma característica puede verse en la parte inferior derecha de la diapositiva.

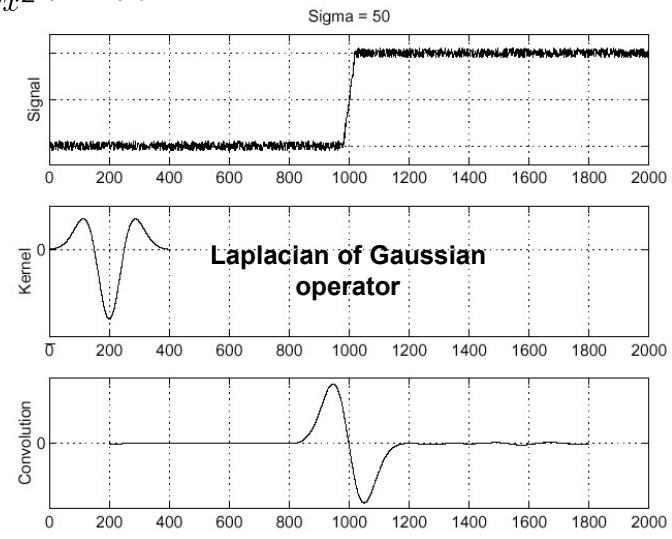
1D Laplacian of Gaussian

Consider $\frac{\partial^2}{\partial x^2}(h \star f)$

f

$\frac{\partial^2}{\partial x^2}h$

$(\frac{\partial^2}{\partial x^2}h) \star f$



Finding these zero-crossings is the goal of the Marr-Hildreth algorithm for edge detection.

Marr, D., & Hildreth, E. (1980). Theory of edge detection. *Procs. of the Royal Society of London*, *207*, 187-217.

Where is the edge? **Zero-crossings of bottom graph**

Slide credit: Steve Seitz

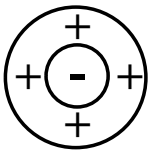
Si partimos de la misma señal ruidosa unidimensional f que vimos en diapositivas anteriores, podemos utilizar directamente la segunda derivada de una gaussiana como filtro. Este operador tiene la forma que vemos en la parte central de la diapositiva.

Al convolucionar directamente la señal con este filtro, combinamos en una única operación el suavizado gaussiano y el cálculo de la segunda derivada. El resultado es una nueva señal en la que, tal y como vimos anteriormente, los bordes pueden localizarse buscando los zero crossings, es decir, los cruces por cero de la respuesta.

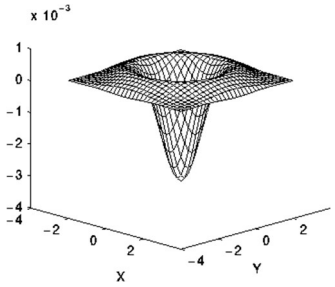
Laplacian of Gaussian

- Example of 3x3 Laplacian kernel:

0	1	0
1	-4	1
0	1	0



"inverted Mexican hat"



- This kernel is not separable, but…
  - we can calculate LoG as the sum of 2D convolutions that are separable
    - Because the Gaussian and its derivatives are!!!
    - In practice, we need 4 1D convolutions (and that is still less computationally expensive than doing a single 2D convolution; unless the kernel is very small)

if the kernel is 7×7 we need 49 multiplications per pixel for the 2D kernel, or 4·7=28 multiplications per pixel for the four 1D kernels; this difference grows as the kernel gets larger

Interesting references about LoG: https://stackoverflow.com/questions/53544983/how-is-laplacian-filter-calculated/53545480#53545480 and https://www.crisluengo.net/archives/1099/

63 of 89

En esta diapositiva podemos ver un ejemplo de kernel laplaciano 3×3, con los valores [0 1 0; 1 -4 1; 0 1 0]. Observamos que sus coeficientes suman cero, como es habitual en los kernels de derivadas. Una forma intuitiva de verlo es que tenemos un valor negativo en el centro rodeado por valores positivos cuya suma compensa exactamente ese valor central. En este sentido, se suele decir que el kernel Laplaciano tiene forma de "sombrero mexicano invertido".

En cualquier caso, dependiendo de la convención de signo, este kernel puede aparecer también multiplicado por -1. En ese caso se intercambian las regiones positivas y negativas, pero la magnitud de la respuesta es exactamente la misma; únicamente cambia el signo.

Este kernel laplaciano no es separable. ¿Por qué? Porque la matriz no tiene rango 1: sus filas no son todas múltiplos unas de otras. Sin embargo, la Laplaciana de la Gaussiana sí puede expresarse como la suma de términos separables.

En términos prácticos, como veremos en las diapositivas siguientes, podemos calcular la LoG mediante cuatro convoluciones 1D y sumar después las dos respuestas obtenidas. Esto sigue siendo computacionalmente más barato que realizar directamente una convolución 2D cuando el kernel es suficientemente grande.

Por ejemplo, con un kernel de 7×7, una convolución 2D directa requiere 49 multiplicaciones y 48 sumas por píxel. Utilizando la descomposición anterior necesitamos cuatro convoluciones 1D de longitud 7, es decir, 4×7 = 28

multiplicaciones por píxel, además de las correspondientes sumas (25, si se incluye la suma de las dos respuestas). La ventaja aumenta a medida que crece el tamaño del kernel. Por ejemplo, si el kernel fuese de 3×3, la ventaja computacional desaparecería. Para aplicar directamente el kernel 2D necesitaríamos 9 multiplicaciones por píxel. En cambio, utilizando la descomposición en cuatro convoluciones 1D de tamaño 3 necesitaríamos 12 multiplicaciones por píxel. Por tanto, en este caso la estrategia separable no supondría una ventaja; de hecho, sería ligeramente más costosa.

Laplacian of Gaussian

The **Laplacian operator** is the **divergence of the gradient**, and is given by the **sum of the second order derivatives** (i.e. the **trace of the Hessian matrix**):

$$\nabla^2 I = \nabla \cdot \nabla I = \begin{bmatrix} \frac{\partial}{\partial x} & \frac{\partial}{\partial y} \end{bmatrix} \begin{bmatrix} \frac{\partial I}{\partial x} & \frac{\partial I}{\partial y} \end{bmatrix}^T = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}$$

$$\frac{\partial^2 (I * G)}{\partial x^2} + \frac{\partial^2 (I * G)}{\partial y^2}$$

Differentiation property of convolution

$$\frac{\partial^2 (f * g)}{\partial x^2} = f * \frac{\partial^2 g}{\partial x^2}$$

$$\frac{\partial^2 (f * g)}{\partial y^2} = f * \frac{\partial^2 g}{\partial y^2}$$

$$I * \frac{\partial^2 G}{\partial x^2} + I * \frac{\partial^2 G}{\partial y^2}$$

Distributive property of convolution

$$f * (g + h) = f * g + f * h$$

$$I * \left(\frac{\partial^2 G}{\partial x^2} + \frac{\partial^2 G}{\partial y^2} \right) = I * \nabla^2 G$$

Laplacian of an image smoothed with a Gaussian kernel

$$\nabla^2 (I * G)$$

Sum of convolutions with the second derivatives of the Gaussian

$$I * \frac{\partial^2 G}{\partial x^2} + I * \frac{\partial^2 G}{\partial y^2}$$

Image convolved with the Laplacian of a Gaussian

$$I * \nabla^2 G$$

64 of 89

Es importante comprender bien cómo se construye la Laplaciana de la Gaussiana porque, de lo contrario, todo lo anterior puede quedarse en generalidades.

El operador laplaciano de una función es la divergencia de su gradiente y, en dos dimensiones, viene dado por la suma de las segundas derivadas respecto a x e y. Es decir, coincide con la traza de la matriz Hessiana. Aplicado a una imagen I, tenemos la segunda derivada respecto a x más la segunda derivada respecto a y.

Ahora bien, en nuestro caso no queremos aplicar simplemente el laplaciano sobre la imagen original, sino sobre una versión suavizada mediante una gaussiana. Por tanto, calculamos el laplaciano de I convolucionada con G: la segunda derivada respecto a x de I*G más la segunda derivada respecto a y de I*G.

Utilizando la propiedad de que la derivada de una convolución puede calcularse derivando cualquiera de sus factores, podemos trasladar esas derivadas al kernel gaussiano. Así obtenemos que el resultado es la imagen convolucionada con la segunda derivada de la gaussiana respecto a x más la imagen convolucionada con la segunda derivada de la gaussiana respecto a y.

Finalmente, utilizando la distributividad de la convolución respecto a la suma, podemos agrupar ambos términos y escribirlo como una única convolución entre la imagen y la suma de las dos segundas derivadas de la gaussiana. Esa suma es precisamente la Laplaciana de la Gaussiana, o LoG.

Laplacian of Gaussian

How do we compute $I * \frac{\partial^2 G}{\partial x^2} + I * \frac{\partial^2 G}{\partial y^2}$ **?**

We know that Gaussian derivatives are separable:

$$\begin{aligned}\frac{\partial G(x,y)}{\partial x} &= -\frac{x}{\sigma^2} \cdot \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2}{2\sigma^2}\right) \exp\left(-\frac{y^2}{2\sigma^2}\right) \\ &= G'_h(x) \cdot G_v(y)\end{aligned}$$

Then:

2nd derivative in X — 2nd derivative in Y

$$\frac{\partial^2 G}{\partial x^2} + \frac{\partial^2 G}{\partial y^2} = G''_h(x) \cdot G_v(y) + G_h(x) \cdot G''_v(y)$$

convolution in one direction with the 2nd derivative of the Gaussian, and then in the other direction, convolution with the Gaussian.

convolution in one direction with the Gaussian and then in the other, convolution with the 2nd derivative of the Gaussian.

65 of 89

Ahora surge la pregunta de cómo calcular, en la práctica, la convolución de la imagen con la segunda derivada respecto a x de la gaussiana más la convolución con la segunda derivada respecto a y.

Para ello volvemos a utilizar la separabilidad de la gaussiana. Sabemos que la gaussiana bidimensional puede escribirse como G(x,y) = G(x)G(y). Por tanto, su segunda derivada respecto a x es G"(x)G(y), porque G(y) no depende de x. Del mismo modo, su segunda derivada respecto a y es G(x)G"(y).

Así, la Laplaciana de la Gaussiana puede escribirse como la suma de dos términos separables: G"(x)G(y) + G(x)G"(y).

Y aquí es donde aparecen explícitamente los cuatro kernels 1D que intervienen en la implementación separable de la Laplaciana de la Gaussiana.

Para calcular el término correspondiente a la segunda derivada respecto a x utilizamos un kernel 1D G"(x) en la dirección horizontal y un kernel gaussiano G(y) en la dirección vertical.

Para el término correspondiente a la segunda derivada respecto a y hacemos justo lo contrario: aplicamos G(x) en horizontal y G"(y) en vertical.

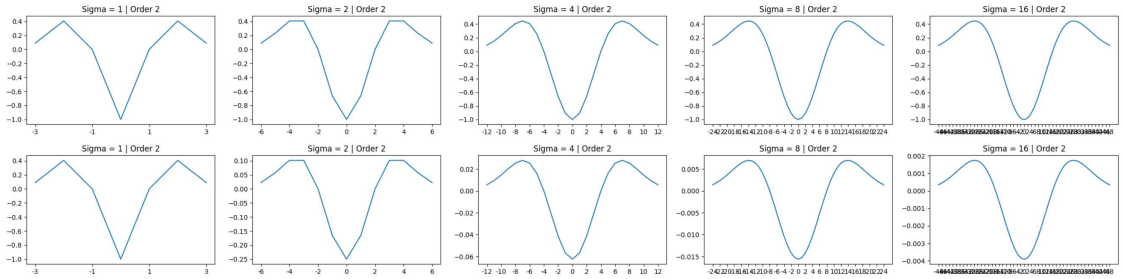
De esta forma, la LoG se obtiene mediante cuatro convoluciones 1D en total, dos para cada uno de los dos términos separables, y finalmente sumando ambas respuestas.

Laplacian of Gaussian

$$L = \sigma^2 \left(G_{xx}(x, y, \sigma) + G_{yy}(x, y, \sigma) \right) \quad \frac{\partial^2 G}{\partial x^2} + \frac{\partial^2 G}{\partial y^2} = G''_h(x) \cdot G_v(y) + G_h(x) \cdot G''_v(y)$$

Why do we multiply by σ^2? 🤔

To **keep response the same (scale-invariant)**, must multiply Gaussian derivative by σ.
Laplacian is the second Gaussian derivative, so it **must be multiplied by σ^2**.



66 of 89

Y ahora entramos en un concepto importante: la normalización de escala. Cuando trabajamos con la Laplaciana de la Gaussiana es habitual multiplicar la suma de las dos derivadas segundas por sigma al cuadrado. La pregunta es: ¿por qué aparece ese factor sigma cuadrado?

En los kernels de suavizado normalizamos para que la suma de los coeficientes sea uno. En los kernels de derivadas la situación es diferente. Cuando aumentamos sigma, no solo estamos analizando estructuras a una escala mayor, sino que también cambia naturalmente la magnitud de las respuestas de las derivadas. Si no hacemos nada, las respuestas obtenidas a distintas escalas no son directamente comparables.

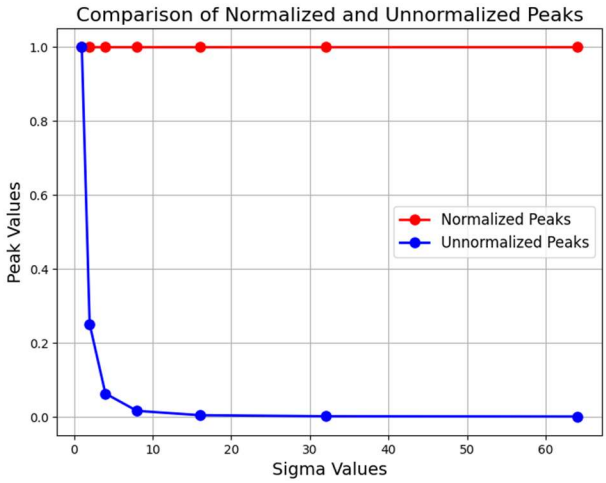
Para compensar este efecto utilizamos la denominada normalización de escala. En su forma habitual, una derivada de orden n se multiplica por sigma elevado a n. Por tanto, para una primera derivada multiplicamos por sigma y para una segunda derivada multiplicamos por sigma al cuadrado. En el caso de la Laplaciana de la Gaussiana obtenemos así sigma² por la suma de las segundas derivadas respecto a x e y. De lo contrario, lo que ocurriría es que, a medida que aumentamos sigma, los coeficientes de los kernels de derivadas se van haciendo cada vez más pequeños, como puede verse en la fila inferior de esta diapositiva. Como consecuencia, también disminuiría la magnitud de la respuesta obtenida al aplicar esos kernels.

La finalidad es compensar el cambio de magnitud introducido por la escala y hacer que las respuestas de las derivadas sean comparables entre diferentes escalas. Esto es especialmente importante cuando queremos detectar estructuras y seleccionar la escala a la que producen una respuesta máxima.

Una intuición visual que puede ayudarnos a entender esta normalización de escala es imaginar un cordón muy largo colocado sobre una mesa con la forma característica de la Laplaciana de la Gaussiana, ese sombrero mexicano invertido. Si aumentamos sigma, la función se hace más ancha. Podemos imaginar que agarramos los extremos del cordón y los vamos separando hacia fuera. Al ensanchar la función, sus lóbulos se extienden espacialmente y, al mismo tiempo, su amplitud disminuye: tanto los lóbulos positivos como el negativo se hacen menos pronunciados. La normalización de escala pretende precisamente compensar esa pérdida de amplitud asociada con el aumento de sigma.

Laplacian of Gaussian

As σ increases, the peak magnitudes (absolute value) of the first and second derivatives of the Gaussian decrease.



67 of 89

Y en esta diapositiva vemos la materialización concreta de esta idea. A medida que aumentamos sigma, la Laplaciana de la Gaussiana se hace más ancha y, si no aplicamos normalización de escala, su valor máximo absoluto va disminuyendo progresivamente.

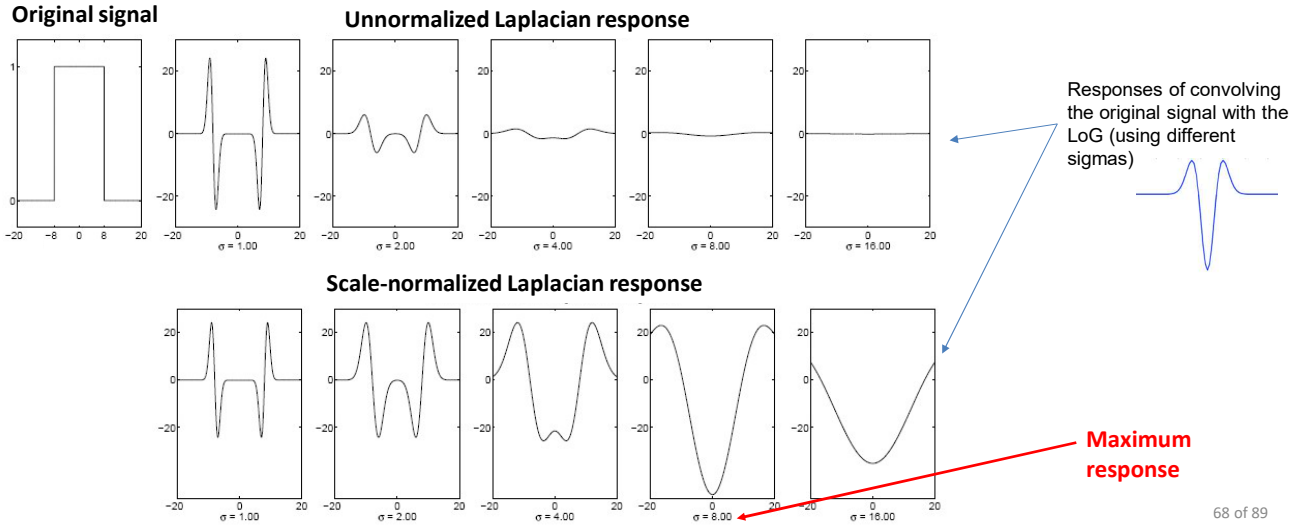
Aquí comparamos precisamente esos picos normalizados y no normalizados. En rojo aparecen los valores correspondientes a la versión normalizada y observamos que se mantienen constantes en torno a 1. En cambio, en la versión no normalizada el pico disminuye de forma muy acusada a medida que aumenta sigma, hasta hacerse prácticamente nulo para valores relativamente grandes, próximos a 10.

Esta gráfica ilustra muy bien por qué introducimos el factor sigma al cuadrado en una derivada de segundo orden: no queremos que la magnitud de la respuesta dependa artificialmente de la escala utilizada, sino poder comparar de forma coherente respuestas obtenidas con diferentes valores de sigma.

En este ejemplo concreto, se está mostrando la segunda derivada de una Gaussiana sin la constante de proporcionalidad (es decir, sin el factor de normalización 1/(sigma sqrt(2pi))).

Laplacian of Gaussian

- The response of a derivative of Gaussian filter to a perfect step edge decreases as σ increases
- To keep response the same (scale-invariant), must multiply Gaussian derivative by σ
- Laplacian is the second Gaussian derivative, so it must be multiplied by σ^2



Esta diapositiva muestra por qué la normalización de escala no es simplemente una cuestión teórica, sino que tiene un efecto directo sobre la respuesta que obtenemos ante una misma estructura de la imagen.

A la izquierda tenemos una señal ideal con forma de escalón. En la fila superior vemos qué ocurre cuando aplicamos la segunda derivada de la gaussiana utilizando valores de sigma cada vez mayores, pero sin normalización de escala. A medida que aumenta sigma, estamos analizando la señal a una escala espacial mayor y la magnitud de la respuesta va disminuyendo. Por tanto, la misma estructura produciría respuestas de distinta magnitud simplemente por haber cambiado la escala del filtro.

En la fila inferior introducimos la normalización de escala. Como estamos trabajando con una derivada de segundo orden, multiplicamos la respuesta por sigma al cuadrado. Esto compensa la disminución de amplitud asociada al aumento de sigma y permite que las respuestas obtenidas a diferentes escalas sean comparables.

La idea general es que para una derivada gaussiana de orden n utilizamos un factor sigma elevado a n. Para una primera derivada sería sigma y para una segunda derivada, como en este caso, sigma al cuadrado. Esto es fundamental cuando queremos comparar la respuesta de una misma estructura a diferentes escalas o determinar a qué escala alcanza una respuesta característica.

Laplacian of Gaussian approximation

- Approximating the Laplacian with a difference of Gaussians:

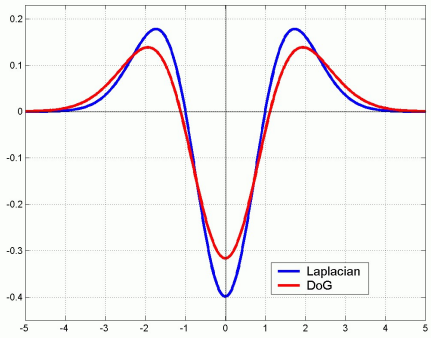
$$L = \sigma^2\left(G_{xx}(x,y,\sigma) + G_{yy}(x,y,\sigma)\right)$$

(Laplacian)

$$DoG = G(x,y,k\sigma) - G(x,y,\sigma)$$

(Difference of Gaussians)

Advantage: we don't even need to implement the second derivative. Just compute the difference of blurred images.



$$G(x,y,k\sigma) - G(x,y,\sigma) \approx (k-1)\cdot L$$

69 of 89

En esta diapositiva introducimos una aproximación muy utilizada de la Laplaciana de la Gaussiana: la diferencia de gaussianas, o Difference of Gaussians, DoG.

Hasta ahora hemos visto que la Laplaciana de la Gaussiana normalizada en escala puede escribirse como sigma al cuadrado multiplicado por la suma de las segundas derivadas de la gaussiana respecto a x e y. Sin embargo, existe una forma de aproximar esta operación sin calcular explícitamente derivadas segundas.

La idea consiste simplemente en tomar dos gaussianas, una con sigma y otra con k por sigma, y calcular su diferencia. Es decir, suavizamos la misma imagen con dos valores distintos de sigma y restamos ambas imágenes suavizadas.

En concreto, G(k sigma) menos G(sigma) es aproximadamente igual a (k-1) multiplicado por sigma al cuadrado por la Laplaciana de G.

En la gráfica de la derecha podemos observar precisamente que ambas funciones presentan una forma muy similar: la DoG aproxima bastante bien la LoG, salvo por el correspondiente factor de escala.

La ventaja práctica es clara: no necesitamos implementar directamente una segunda derivada. Basta con realizar dos suavizados gaussianos a escalas diferentes y restar los resultados.

Effect of σ on derivatives (scales)



σ = 1 σ = 3

The apparent structures differ depending on Gaussian's scale parameter.

Larger values: larger scale edges detected.
Smaller values: finer features detected.

Kristen Grauman, UT-Austin

Esta diapositiva muestra de forma muy intuitiva el efecto que tiene sigma sobre las estructuras que detectamos mediante derivadas gaussianas.

A la izquierda tenemos la imagen original. En el centro vemos la respuesta obtenida con sigma igual a 1. Como el suavizado es relativamente pequeño, se conservan y aparecen muchas estructuras finas: detalles del pelo, textura del fondo y bordes pequeños.

A la derecha utilizamos sigma igual a 3. Al aumentar sigma, el suavizado previo es mayor y desaparecen muchas de esas estructuras de alta frecuencia. Como consecuencia, la respuesta se concentra en bordes más gruesos y estructuras de mayor escala, como el contorno general del panda.

Esta es precisamente la interpretación del parámetro sigma como parámetro de escala. Valores pequeños de sigma permiten detectar detalles finos y estructuras pequeñas. Valores grandes de sigma eliminan esos detalles y permiten centrarnos en estructuras más grandes.

Por tanto, no existe necesariamente un único sigma correcto para una imagen: depende de la escala de las estructuras que queramos analizar.

Summary: Two types of convolution kernels

Smoothing

$$\sum g_i = 1$$

(Smoothing a constant function should not change it)

Example: (1/4) * [1 2 1]

Low-pass filter (Gaussian)

Differentiating

$$\sum g_i = 0$$

(Differentiating a constant function should return 0)

Example: [-1 0 1]

High-pass filter (derivative of Gaussian)

S. Birchfield, Clemson Univ., ECE 847, http://www.ces.clemson.edu/~stb/ece847

Esta diapositiva resume bien dos grandes familias de kernels que hemos ido utilizando: los kernels de suavizado y los kernels de derivación.

En el caso de los filtros de suavizado, los coeficientes deben sumar uno. La intuición es sencilla: si aplicamos el filtro sobre una región de intensidad constante, esa intensidad debería mantenerse. Por eso normalizamos, por ejemplo, un kernel como [1, 2, 1] dividiendo por 4. Este tipo de filtros actúa como paso bajo, porque atenúa las variaciones rápidas de intensidad y, por tanto, las altas frecuencias.

En cambio, en los filtros de derivación los coeficientes deben sumar cero. La razón es que la derivada de una función constante debe ser cero. Un ejemplo muy sencillo es el kernel [-1, 0, 1], que responde a cambios de intensidad pero devuelve cero en regiones uniformes. Estos filtros tienen un comportamiento de paso alto, porque resaltan precisamente las variaciones rápidas de la señal.

Summary: Two types of convolution kernels

Two closely related problems:



blur
(to remove noise)



differentiate
(to highlight details)

S. Birchfield, Clemson Univ., ECE 847, http://www.ces.clemson.edu/~stb/ece847

Summary: Two types of convolution kernels

Two closely related problems:





blur
(to remove noise)

differentiate
(to highlight details)

Underlying math is the same!

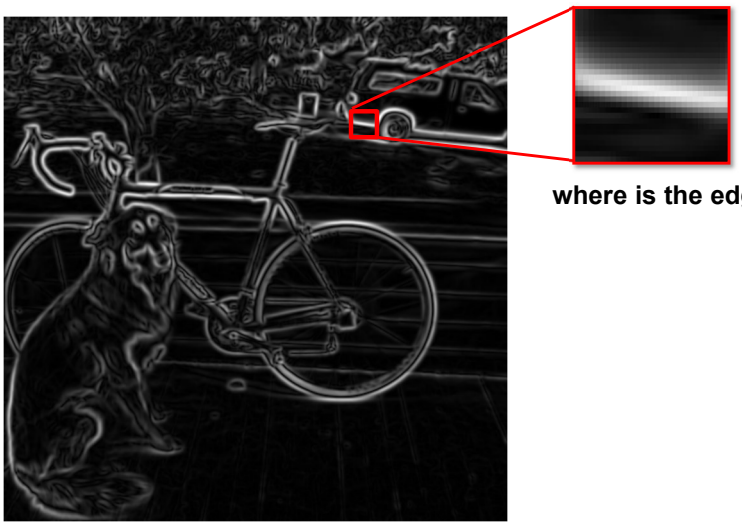
S. Birchfield, Clemson Univ., ECE 847, http://www.ces.clemson.edu/~stb/ece847

A la izquierda tenemos la versión suavizada de la imagen, donde se han eliminado muchas pequeñas variaciones y ruido. A la derecha vemos una representación basada en derivadas, en la que aparecen resaltados los cambios de intensidad y, por tanto, los contornos y detalles estructurales de la imagen.

La idea importante es que, aunque visualmente los resultados parezcan muy distintos, desde el punto de vista matemático ambas operaciones se implementan de una forma muy similar: mediante convoluciones con distintos kernels. En un caso utilizamos un kernel de suavizado y en el otro, de una u otra forma, un kernel de derivadas.

Canny Edge Detector

- Sometimes we need to accurately locate the boundary….



Aquí introducimos el detector de bordes de Canny. Hasta ahora hemos visto cómo calcular derivadas y cómo obtener una medida de la magnitud del gradiente, pero eso por sí solo no nos da necesariamente una localización precisa del borde.

En la imagen principal vemos precisamente este problema. La respuesta del gradiente puede ocupar una banda relativamente ancha alrededor de una transición. Si ampliamos una de esas zonas, observamos que no aparece una línea fina, sino una región con distintos valores de respuesta. Entonces surge la pregunta: ¿dónde está exactamente el borde dentro de esa banda?

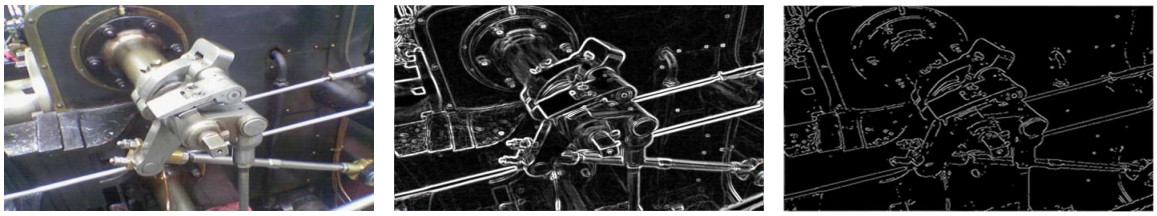
La idea fundamental de Canny es transformar esa respuesta gruesa en una localización mucho más precisa del borde. Para ello no se limita a calcular la magnitud del gradiente, sino que incorpora pasos adicionales, que veremos a continuación.

Canny Edge Detector

- Canny, J., "A Computational Approach To Edge Detection", IEEE Trans. on Pattern Analysis and Machine Intelligence, 8(6):679–698, 1986 (~50K citations in Google Scholar)
- Still popular because
  - easy to implement
  - small number of intuitive parameters
  - computationally efficient
  - good results

Python:
https://docs.opencv.org/4.8.0/da/d22/tutorial_py_canny.html

Original Image **Sobel Magnitude** **Canny Edge Detector**



75 of 89

El detector de bordes de Canny fue propuesto por John Canny en 1986 en el artículo "A Computational Approach to Edge Detection". A pesar de ser un método clásico, sigue utilizándose bastante porque combina varias ventajas: es sencillo de implementar, tiene pocos parámetros (y estos son bastante intuitivos), es computacionalmente eficiente y produce resultados razonablemente buenos.

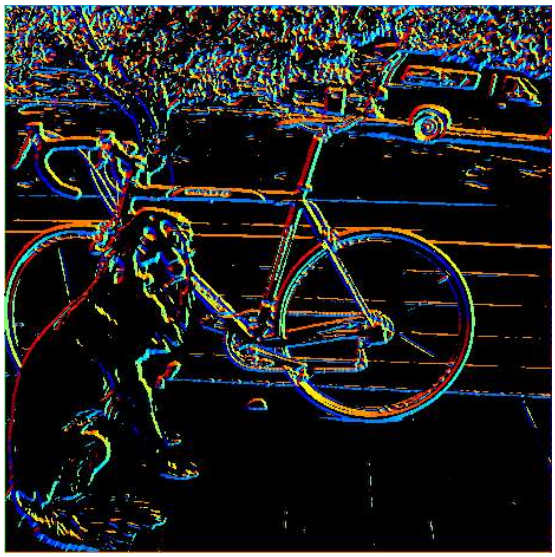
En la parte inferior podemos comparar tres cosas. A la izquierda está la imagen original. En el centro aparece la magnitud del gradiente obtenida con Sobel. Como veíamos en la diapositiva anterior, esta respuesta suele producir bordes relativamente gruesos, y muchas respuestas debidas a textura o pequeñas variaciones locales.

A la derecha aparece el resultado de Canny. Aquí los bordes son mucho más finos y selectivos. Esto se debe a que Canny no se limita a calcular el gradiente, sino que añade pasos posteriores para refinar la localización de los bordes y eliminar respuestas débiles o poco consistentes.

La idea importante es que Canny no es simplemente "otro kernel", sino un procedimiento completo de detección de bordes compuesto por varias etapas.

Canny Edge Detector

1) Filter image with derivative of Gaussian and compute the gradient (magnitude and orientation) at each pixel



theta = atan2(gy, gx)



Aquí empieza el primer paso del detector de Canny. Lo primero que hacemos es calcular el gradiente de la imagen, normalmente después de aplicar un suavizado gaussiano para reducir la influencia del ruido. En la práctica, esto puede implementarse mediante un suavizado seguido de operadores como Sobel.

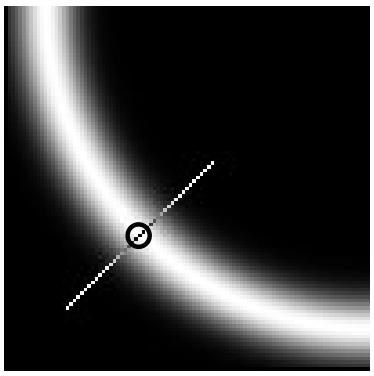
En cada píxel obtenemos dos componentes, Gx y Gy. A partir de ellas calculamos, por un lado, la magnitud del gradiente, que nos indica la intensidad del cambio local, y, por otro, su orientación mediante atan2(Gy,Gx), que nos indica la dirección en la que aumenta más rápidamente la intensidad.

En esta visualización el color codifica precisamente ese ángulo de orientación del gradiente. Por tanto, píxeles con colores diferentes corresponden a direcciones diferentes del gradiente. Esta orientación va a ser fundamental en el siguiente paso de Canny, porque nos permitirá decidir en qué dirección debemos buscar si un píxel es realmente un máximo local.

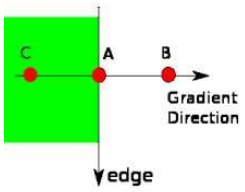
Canny Edge Detector

2) Non-maximum suppression

A full scan of the image is done to remove any unwanted pixels which may not constitute the edge.



For each pixel, we keep it only if it has the highest gradient magnitude along the gradient direction in its local neighborhood.
This makes the edges thinner (1 pixel wide).



Since pixel A has orientation 0º → we check its left (C) and right (B) neighbors

- We check whether each pixel is a **local maximum in its neighborhood along gradient direction**
  - Point A is on the edge. Gradient direction is normal to the edge. Point B and C are in the gradient direction.
  - Point A is checked with point B and C to see if it forms a local maximum.
  - If so, it is considered for next stage, otherwise, it is suppressed (put to zero).

https://docs.opencv.org/4.x/da/d22/tutorial_py_canny.html

Aquí tenemos el segundo paso fundamental del detector de Canny: la supresión de no máximos, o non-maximum suppression.

La idea es que, después de calcular la magnitud y la orientación del gradiente, todavía podemos tener una banda relativamente gruesa de respuesta alrededor de un borde. Lo que queremos ahora es adelgazar esa banda y quedarnos únicamente con el punto que mejor representa la localización del borde.

Para cada píxel candidato, miramos la dirección del gradiente, que es perpendicular al borde. En esa dirección comparamos la magnitud del gradiente del píxel actual con la de los dos puntos situados a ambos lados. Si el píxel actual es un máximo local, es decir, si su respuesta es mayor que la de esos dos vecinos en la dirección del gradiente, lo conservamos. Si no lo es, lo suprimimos, normalmente poniéndolo a cero.

En el pequeño esquema de la derecha, el punto A se compara con B y C porque esos puntos están alineados con la dirección del gradiente. Si A tiene una magnitud mayor que ambos, se mantiene; en caso contrario, se elimina.

El resultado es que los bordes pasan de ser bandas relativamente anchas a líneas de un píxel de grosor.

En implementaciones reales, no se trabajan todas las orientaciones posibles de forma literal. El problema es que el gradiente puede tener cualquier orientación: 17°, 63°, 128°, etc. Pero en una cuadrícula de píxeles no tienes, en general, dos píxeles vecinos que estén exactamente alineados con una dirección arbitraria como

17°. Lo que se hace es cuantizar la dirección del gradiente a una de unas pocas direcciones representativas, normalmente 0°, 45°, 90° y 135°.

Before Non-max Suppression



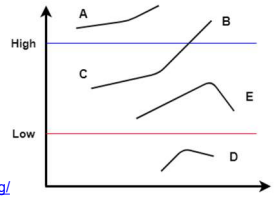
After Non-max Suppression



Canny Edge Detector

3) Linking and thresholding edges (hysteresis)

- We only want strong edges
- 2 thresholds (high, low), 3 cases (R: response):
  - R > high: strong edge!
  - R < low: no edge
  - R < high but R > low: weak edge
    - **Weak edges are edges iff they connect to strong ones**
    - Look in some neighborhood (usually 8 closest)





https://theailearner.com/tag/hysteresis-thresholding/

80 of 89

Aquí tenemos el tercer paso de Canny: el umbralizado con histéresis y el enlazado de bordes.

Después de la supresión de no máximos ya tenemos bordes finos, pero todavía aparecen muchas respuestas débiles. Algunas corresponden a bordes reales y otras pueden deberse a ruido o pequeñas variaciones locales. Para decidir cuáles conservar utilizamos dos umbrales: uno alto y otro bajo.

Si la respuesta de un píxel está por encima del umbral alto, lo consideramos un borde fuerte y lo conservamos. Si está por debajo del umbral bajo, lo descartamos. El caso interesante es cuando la respuesta queda entre ambos umbrales: hablamos entonces de un borde débil.

Un borde débil se conserva únicamente si está conectado con un borde fuerte o con una cadena de bordes débiles que finalmente conecta con uno fuerte. Si no existe esa conexión, se elimina.

Esta es precisamente la idea de la histéresis: no tomamos la decisión utilizando un único umbral independiente para cada píxel, sino que utilizamos también la conectividad espacial. De esta manera podemos conservar fragmentos débiles que forman parte de un borde real y, al mismo tiempo, eliminar muchas respuestas débiles aisladas debidas al ruido.

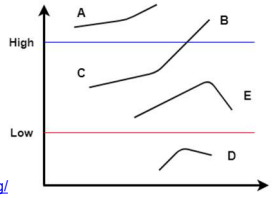
La pequeña gráfica de la parte inferior ilustra los tres casos: respuestas

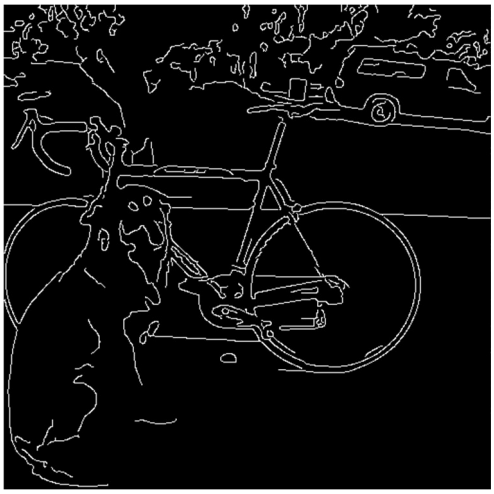
por encima del umbral alto (nos quedamos con ellas), respuestas por debajo del umbral bajo (se ignoran/eliminan) y respuestas intermedias (cuya aceptación depende de si están conectadas con un borde fuerte).

Canny Edge Detector

3) Linking and thresholding edges (hysteresis)

- We only want strong edges
- 2 thresholds (high, low), 3 cases (R: response):
  - R > high: strong edge!
  - R < low: no edge
  - R < high but R > low: weak edge
    - **Weak edges are edges iff they connect to strong ones**
    - Look in some neighborhood (usually 8 closest)





https://theailearner.com/tag/hysteresis-thresholding/



Canny Edge Detector

0. Many times we first smooth the image to reduce the impact of noise



1. Filter image with derivative of Gaussian and find magnitude and orientation of gradient



2. Non-maximum suppression



3. Linking and thresholding (hysteresis):
   - Define two thresholds: low and high
   - Use the high threshold to start edge curves and the low threshold to continue them

Source: D. Lowe, L. Fei-Fei, J. Redmon

Esta diapositiva resume el pipeline completo del detector de Canny.

El paso 0 consiste en suavizar la imagen para reducir la influencia del ruido. Esto es importante porque las derivadas son muy sensibles a pequeñas variaciones locales y podrían generar muchas respuestas espurias.

En el paso 1 calculamos el gradiente de la imagen. Para cada píxel obtenemos la magnitud del gradiente y su orientación.

En el paso 2 aplicamos non-maximum suppression. Utilizamos la orientación del gradiente para comparar cada píxel con los puntos situados a ambos lados en esa dirección y conservar únicamente los máximos locales. De esta forma adelgazamos las bandas de respuesta y obtenemos bordes de un píxel de grosor.

Finalmente, en el paso 3 aplicamos thresholding con histéresis. Definimos dos umbrales, uno alto y otro bajo. Los píxeles por encima del umbral alto se consideran bordes fuertes. Los píxeles con respuesta intermedia se mantienen únicamente si están conectados, directa o indirectamente, con un borde fuerte. Los que quedan por debajo del umbral bajo se eliminan.

La idea de la frase final es precisamente esa: el umbral alto sirve para identificar los bordes en los que tenemos mucha confianza, y el umbral bajo permite continuar esos bordes a través de zonas donde la respuesta es más débil.

Canny Edge Detector



original | Canny with $\sigma_d = 1$ | Canny with $\sigma_d = 2$

Parameters to tune:

high threshold
low threshold
σ_d: width of the Gaussian derivative filter
σ_s: width of the Gaussian smoothing filter

- The choice of σ_d depends on desired behavior
  - Large σ_d detects "large-scale" edges
  - Small σ_d detects fine edges

Adapted from S. Seitz

Esta diapositiva resume los principales parámetros que controlan el comportamiento del detector de Canny y muestra cómo afectan al resultado.

Por un lado, tenemos los dos umbrales utilizados en la histéresis: el umbral alto determina qué respuestas consideramos bordes fuertes y el umbral bajo permite decidir hasta qué punto seguimos esos bordes a través de respuestas más débiles.

Por otro lado, aparecen los parámetros relacionados con la escala. En particular, sigma controla el ancho de la gaussiana o de la derivada de gaussiana que utilizamos. Un sigma pequeño implica trabajar a una escala fina: se preservan más detalles y aparecen bordes correspondientes a estructuras pequeñas. Un sigma mayor introduce más suavizado y hace que desaparezcan muchos detalles finos, de modo que permanecen principalmente estructuras de mayor escala.

Esto se aprecia claramente en las imágenes superiores. Con sigma igual a 1 aparecen muchos más contornos y detalles locales. Cuando aumentamos sigma a 2, algunas de esas respuestas desaparecen y el resultado se concentra en estructuras más grandes y persistentes.

Por tanto, los parámetros de Canny no determinan únicamente cuántos bordes detectamos, sino también la escala espacial de las estructuras que consideramos relevantes.

Canny Edge Detector



original image



high threshold
(strong edges)



low threshold
(weak edges)



hysteresis threshold

Source: L. Fei-Fei

84 of 89

A la izquierda vemos el resultado si utilizamos únicamente un umbral alto. En este caso nos quedamos solamente con las respuestas más fuertes. El resultado es bastante limpio, pero algunos bordes reales quedan fragmentados o directamente desaparecen porque parte de su respuesta no alcanza ese umbral.

En el centro vemos el caso contrario: utilizar únicamente un umbral bajo. Ahora conservamos muchas más respuestas, incluidas zonas débiles que pueden corresponder a bordes reales. El problema es que también aparecen muchas respuestas poco relevantes o debidas a ruido.

A la derecha aplicamos la estrategia de histéresis. Los píxeles por encima del umbral alto se aceptan directamente como bordes fuertes. Los que están entre el umbral bajo y el alto solo se mantienen si están conectados con esos bordes fuertes. De esta forma podemos recuperar continuidad en los contornos sin introducir tantas respuestas espurias como cuando utilizamos únicamente el umbral bajo.

La idea importante es que la histéresis combina lo mejor de ambos mundos: la fiabilidad del umbral alto y la capacidad del umbral bajo para prolongar bordes reales a través de zonas donde la respuesta es más débil.

Homework

Let's imagine a 2D Gaussian kernel of size $9x9$.

- What would the equivalent sigma be (assuming that k has to be the smallest integer greater than or equal to 3σ)?
- If we wanted to apply this same $9x9$ kernel 3 consecutive times, what would the equivalent kernel size be?

  Note: When you apply the same kernel of size N a total of k times, the equivalent filter size is $N + (k-1) \cdot (N-1)$
- If we wanted to apply this same $9x9$ kernel 3 consecutive times, what would the equivalent kernel size be using the property that the Gaussian is closed under convolution (i.e., convolving N consecutive times with sigma is equal to a single convolution with a Gaussian with $\sigma\sqrt{N}$)? In this case, we look for the filter with the equivalent blurring effect.

Homework (exam quizzes, in Spanish)

¿Qué propiedad o teorema relativo a la convolución es el que, principalmente, permite la creación/composición de filtros más complejos a partir de filtros más sencillos?

a) Asociativa
b) Conmutativa
c) Distributiva
d) Teorema de la Convolución

¿Qué tipo de filtro considera que es el siguiente?

$$\begin{matrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{matrix}$$

a) Suavizado Gaussiano
b) Laplaciana de la Gaussiana
c) Sharpening
d) Ninguna de las anteriores

Homework (exam quizzes, in Spanish)

Si suavizamos una imagen tres veces consecutivas con un filtro Gaussiano σ, ¿cuál sería el sigma equivalente que nos permitiría obtener el mismo resultado suavizando una única vez?

a) $\sigma\sqrt{3}$
b) $\sqrt{3\sigma}$
c) $3\sqrt{\sigma}$
d) Ninguna de las anteriores

¿Cuál de las siguientes afirmaciones sobre los algoritmos de Canny y Otsu es correcta?

a) El detector de bordes de Canny se comporta como un filtro lineal dado que emplea filtrado Gaussiano para suavizar/emborronar la imagen y, a continuación, emplea filtrado lineal para calcular el gradiente.
b) El umbral óptimo en el algoritmo de Otsu se obtiene minimizando la varianza inter-clase o maximizando la varianza intra-clase.
c) En el algoritmo de Canny obtendremos bordes más discontinuos/fragmentados si reducimos el umbral bajo.
d) Las tres respuestas anteriores son erróneas.

Homework (exam quizzes, in Spanish)

¿Cuál de las afirmaciones siguientes sobre procesado de imagen es cierta?

a) Las derivadas en X detectan bordes horizontales, y las derivadas en Y detectan bordes verticales.

b) La magnitud del gradiente puede tener valores negativos, dado que el gradiente puede hacer referencia a pendientes positivas o negativas.

c) El gradiente en una imagen es mayor en las regiones planas que en las zonas con bordes.

d) Ninguna de las anteriores es correcta.

¿Cuáles de las siguientes afirmaciones relativas a la operación de convolución son correctas?

a) La convolución es equivariante con la traslación.

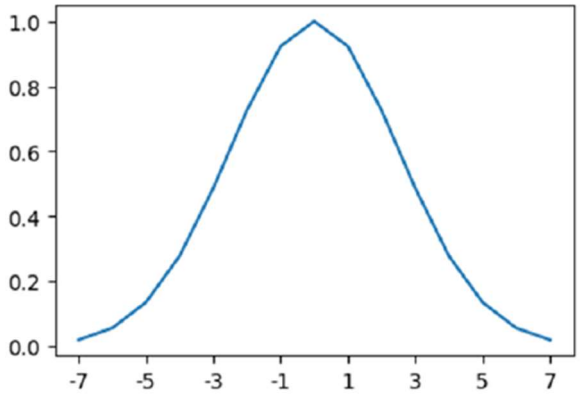
b) La convolución no es equivariante con el escalado.

c) La convolución no es equivariante con la rotación.

d) Todas las anteriores son correctas.

Homework (exam quizzes, in Spanish)

La siguiente figura representa el perfil de un kernel 1D. ¿Qué tipo de filtro representa?



a) Un kernel de suavizado Gaussiano.

b) Un kernel de primeras derivadas.

c) Un kernel de segundas derivadas.

d) Ninguno de los anteriores.

Gaussian Filtering and Edges

Pablo Mesejo

pmesejo@go.ugr.es

Universidad de Granada

Departamento de Ciencias de la Computación e Inteligencia Artificial

